

Flow With What You Know

顺着你已知的东西而“流”

We provide an accessible introduction to flow-matching and rectified flow models, which are increasingly at the forefront of generative AI applications. Typical descriptions of them are often laden with extensive probability-math equations, which can form barriers to the dissemination and understanding of these models. Fortunately, before they were couched in probabilities, the mechanisms underlying these models were grounded in basic physics, which provides an alternative and highly accessible (yet functionally equivalent) representation of the processes involved.

本文为流匹配（flow matching）与 rectified flow 模型提供一份通俗易懂的入门介绍——这类模型正日益站到生成式 AI 应用的最前沿。常见的介绍往往堆满概率数学公式，反而成为理解与传播这些模型的障碍。所幸的是，在它们被概率语言包裹之前，其内在机制本就扎根于基础物理；物理视角为此提供了一种替代的、且高度易于理解（在功能上等价）的表述方式。

Author 作者	Scott H. Hawley
Affiliation 单位	Belmont University and Hyperstate Music, Inc.
Published 发布	April 28, 2025
Venue 出处	ICLR Blogposts 2025
Original 原文	d2jud02ci9yv69.cloudfront.net/.../flow-with-what-you-know/
Code 代码	Colab notebook （可执行版）

关于这份 PDF —— 由原文离线归档并重新排版而成，方便阅读与学习：

1. 中英对照：每段先给出英文原文（黑色），紧随其后是中文翻译（蓝灰色，带左侧竖线）。脚注同样双语给出。
2. 图与公式：原文 11 张插图与风场图已全部离线嵌入；两处公式按原文排版重现。
3. 动画：原文的 3 段 MP4 动画在 PDF 中无法播放，已改为按时间等间隔抽取 6 帧的序列图（图 8、图 15、图 16），并附说明。
4. 代码：原文折叠的代码块统一移入文末 附录 A，正文保持叙述连贯；代码为原文原样，未作改动。
5. 引用：正文中的上标 ^[n] 与文末参考文献编号一一对应，按首次出现顺序编号。

目录 Contents

1 Introduction 引言

1.1 What's a Flow? 什么是「流」？

2 How Do FM/RF Models Work? FM/RF 模型是如何工作的？

2.1 The Starting Setup 初始设定

2.2 How Are Flows “Learned”? 流是如何被「学习」的？

2.2.1 The Neural Network's Job 神经网络的任务

2.3 Training Code 训练代码

3 “Reflow” to Go Straighter & Faster 「Reflow」：让轨迹更直、更快

3.1 Upgrading Our Gear First 先升级我们的装备

3.1.1 More Points Where Needed (via Time Warping) 在需要处加密采样点（时间扭曲）

3.1.2 Better Integration / Sampling 更好的积分 / 采样格式

3.2 Learning to “Reflow” 学习「Reflow」

3.2.1 Train the Reflowed Model 训练 Reflow 模型

4 Connecting with Other Models 与其他模型的联系

4.1 From Dots to Images, Audio, etc.. 从二维点到图像、音频.....

4.2 Diffusion Models 扩散模型

4.3 Optimal Transport 最优传输

4.4 Normalizing Flows 归一化流

5 Summary 总结

References 参考文献

Appendix A — Full Code 附录 A：完整代码

1 Introduction

引言

Flow-based generative AI models [1], [2] have been gaining significant traction as alternatives or improvements to traditional diffusion approaches in image and audio synthesis. These flow models excel at learning optimal trajectories for transforming probability distributions, offering a mathematically elegant framework for data generation. The approach has seen renewed momentum following Black Forest Labs' success with their FLUX models [3], spurring fresh interest in the theoretical foundations laid by earlier work on Rectified Flows [2] in ICLR 2023. Subsequent improvements [4] have even reached the level of state-of-the-art generative models for one or two-step generation.

基于流的生成式 AI 模型 [1,2] 正日益受到关注，被视为图像与音频合成中传统扩散方法的有力替代或改进。这类流模型擅长学习变换概率分布的最优轨迹，为数据生成提供了一个数学上优雅的框架。在 Black Forest Labs 的 FLUX 模型 [3] 取得成功之后，这一路线重获动力，也重新激发了人们对 2023 年 ICLR 上 Rectified Flow [2] 早期工作所奠定的理论基础的兴趣。随后的改进 [4] 甚至已达到「一步或两步生成」的最先进水平。

Intuitively, these models operate akin to the fluid processes that transform the shapes of clouds in the sky. While recent expositions [5], [6] have attempted to make these concepts more accessible through probability theory, the underlying physical principles may offer a more direct path to understanding for many readers. By returning to the basic physical picture of flows that inspired these generative models, we can build both intuition and deep understanding - insights that may even guide the development of new approaches.

直观上，这类模型的运作方式类似于天空中云的形状发生变化时所遵循的流体过程。虽然近来的若干讲解 [5,6] 试图通过概率论让这些概念更易理解，但对许多读者而言，其背后的物理原理可能是一条更直接的理解路径。回到当初启发这类生成模型的最基本物理图景——「流」，我们既能建立直觉，也能获得深入的理解；这些洞见甚至可能指引新方法的诞生。

1.1 What's a Flow?

什么是「流」？

In the real world, things typically follow curved paths - like water flowing in a river, or crowds of people navigating around obstacles. Here's map of wind provided from the [WW2010 atmospheric science project at UIUC](#): at every point in space, the wind has a velocity vector, and the air moves along "streamlines" or "trajectories" parallel to the velocity vectors...

在现实世界中，事物通常沿曲线路径运动——比如河流中的水，或绕过障碍物行进的人群。下图是 UIUC 的 WW2010 大气科学项目给出的风场图：空间中每一点上，风都有一个速度矢量，空气则沿着与这些速度矢量平行的「流线」（streamline）或「轨迹」（trajectory）运动.....

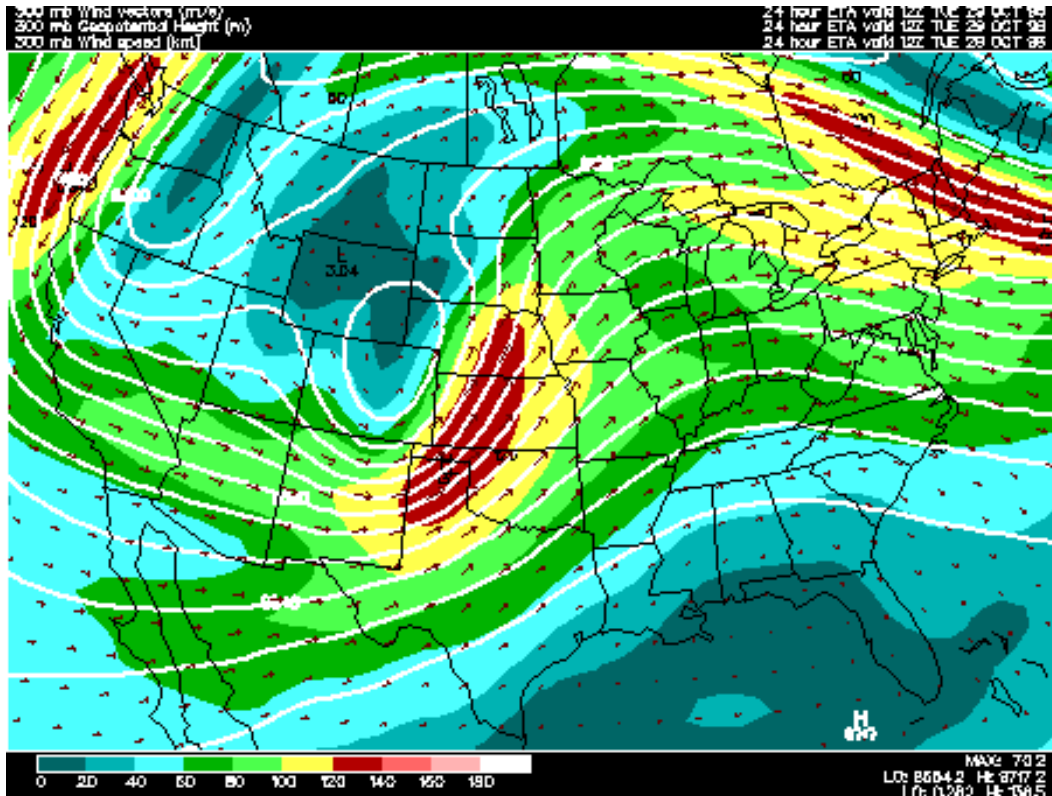


图 1 Wind map from the WW2010 atmospheric science project at UIUC. At every point in space the wind has a velocity vector; air moves along streamlines parallel to those vectors.

UIUC 的 WW2010 大气科学项目给出的风场图。空间中每一点上，风都有一个速度矢量；空气沿着与这些矢量平行的流线运动。

Notice that the streamlines never cross. If the streams were to cross... “it would be bad.”¹ That would imply that the velocity at some point is undefined.

注意，流线从不相交。如果流线相交.....「那就糟了。」¹那意味着某一点的速度是没有定义的。

注 1 "Don't cross the streams." -- Ghostbusters (1984)

「不要跨越流束。」——《捉鬼敢死队》(1984)

Note: 注意

This non-crossing property is what allows these flows to be invertible (i.e., reversible), a property you sometimes hear in isolation when reading more formal descriptions of flow models.

正是这种不相交的性质，使得这类流是可逆的（invertible，即一一对应、可反演）——在阅读更为形式化的流模型描述时，你有时会单独看到「可逆性」这一说法。

So, at every point in space there's a velocity vector telling the little bits of fluid where to go. And just like water or wind flows may depend not only on spatial position but also time, so too can our velocity vector field depend on position and time.

因此，空间中每一点都有一个速度矢量，告诉流体微元该往哪儿走。而且，正如水流或风场不仅依赖空间位置、还可能随时间变化，我们的速度矢量场同样可以同时依赖位置与时间。

Flow matching learns these natural paths by focusing on the *velocity* at each point - essentially asking, “Which way should each data point be moving at this moment?”

流匹配（flow matching）通过关注每一点上的速度来学习这些自然路径——本质上就是在问：「此刻，每个数据点应该朝哪个方向运动？」

Terminology: "FM/RF", "Rectified" 术语说明：「FM/RF」与「Rectified」

It may seem confusing to see "flow matching" and "rectified flows" sometimes used interchangeably, but this is because *they are the same*^[7]. In this blog post, we'll use the collective term "FM/RF" models.

Also, note that *there is no explicit "rectification" mechanism* in rectified flows; rather, "rectified" is a description of the effect of flow-matching, i.e., of transforming crossing trajectories to non-crossing ones. The addition of "Reflow" to the rectified flow paper^[2] is a powerful extension we will cover further below.

看到「flow matching」与「rectified flows」有时被混用可能会让人困惑，但它们其实是同一个东西。本文统一使用「FM/RF 模型」这一合称。另外请注意：rectified flow 中并不存在某种显式的「矫正（rectification）」机制；「rectified（被矫正）」只是对流匹配效果的一种描述，即把相交的轨迹变换为不相交的轨迹。而 rectified flow 论文中额外提出的「Reflow」则是一个强有力的扩展，我们将在后文介绍。

2 How Do FM/RF Models Work?

FM/RF 模型是如何工作的？

To gain a deep understanding of how models work, having an executable toy model is often a key instructional tool. This tutorial is written as an executable Jupyter notebook¹, though you can make sense of it without the code, so we will typically collapse or hide the code. But if you want to see it, you can expand the drop-down arrows.

要想深入理解一个模型如何工作，有一个可以实际运行的玩具模型往往是最关键的教学工具。本教程就是以
一个可执行的 Jupyter notebook 形式写成的^[Colab]；不过即使不看代码也能理解内容，所以我们通常会
把代码折叠或隐藏起来；如果你想看，展开下拉箭头即可。

注 1 Colab link: <https://tinyurl.com/iclr-2025-flow-blogpost-colab>

For instance, the code starts with importing packages...

例如，代码一开始是导入各种包.....

[代码 1: Installs & imports] → 见附录 A

Choose Your Own Data Shapes 自选数据形状

The [executable notebook version of this lesson](#) lets you choose various shapes to “morph” between. For this reading, we’ll go from a Gaussian to a Spiral, but other choices are available in the notebook.

本课的可执行 notebook 版本允许你选择要在其间「变形（morph）」的各种形状。在本篇中，我们从高斯分布出发走向螺旋，notebook 中还有其他选项可供选择。

[代码 2: snippet] → 见附录 A

Note: 注意

Flow models don't require gaussian prior distributions. You can choose whatever you want.

流模型并不要求先验分布必须是高斯分布——你可以任选。

With the imports in place and the choice of starting and ending distributions made, we’re ready to define some utilities to generate and visualize our data. Let’s take a look:

导入就位、起始与终止分布也已选定，接下来就可以定义一些工具函数来生成并可视化数据。先来看一看：

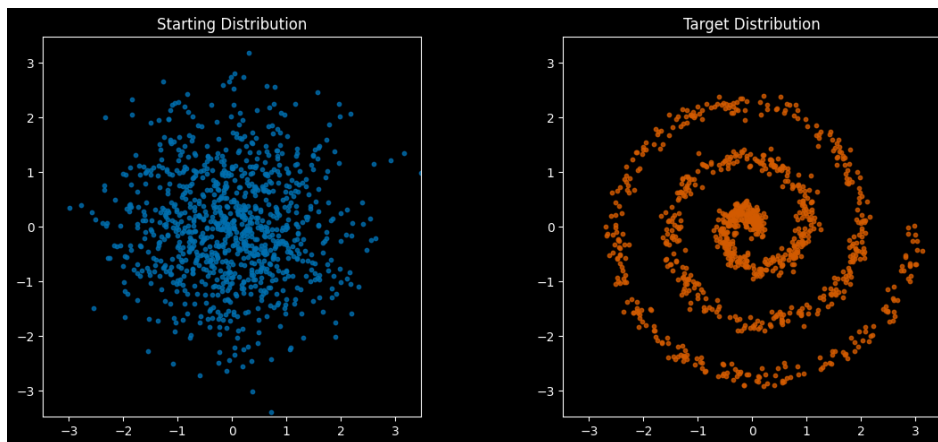


图 2 The source (Gaussian) and target (spiral) distributions.
源分布（高斯）与目标分布（螺旋）。

The process of transitioning from the starting “source” to the final “target” might include snapshots like these:

从起始的「源（source）」过渡到最终的「目标（target）」，其中的快照可能长这样：

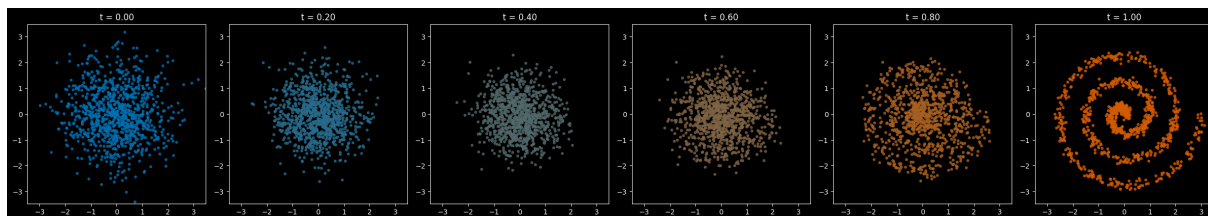


图 3 Snapshots of the transition from the source distribution to the target distribution.
从源分布过渡到目标分布的若干快照。

(Note the colors aren’t meaningful, they’re just added to make it easier to distinguish what we’re looking at. Our data are just points in 2-D space.)

（注意：颜色没有实际含义，只是为了更容易区分我们在看什么。我们的数据就是二维空间中的一些点。）

So, how do we get the points from the source distribution to fit with the target distribution? The simplest way (though not the only way) is to assume points move in straight lines from source to target. Even though our network might learn more complex paths later, this gives us a starting point for training.

那么，如何让这些点从源分布变成符合目标分布的样子？最简单的方式（并非唯一方式）是假设每个点都从源沿直线运动到目标。尽管网络之后可能会学到更复杂的路径，但这至少给了我们一个训练的起点。

2.1 The Starting Setup

初始设定

The training setup for flow matching models is as follows:

流匹配模型的训练设定如下：

1. We start by *randomly pairing* points from the source & the target – yes, really. 🎲
2. We move the points along straight trajectories, and the speed of each point is constant.
1. 我们从源与目标中随机地配点——没错，真的是随机配对。🎲
2. 让这些点沿直线轨迹运动，且每个点的速度恒定。

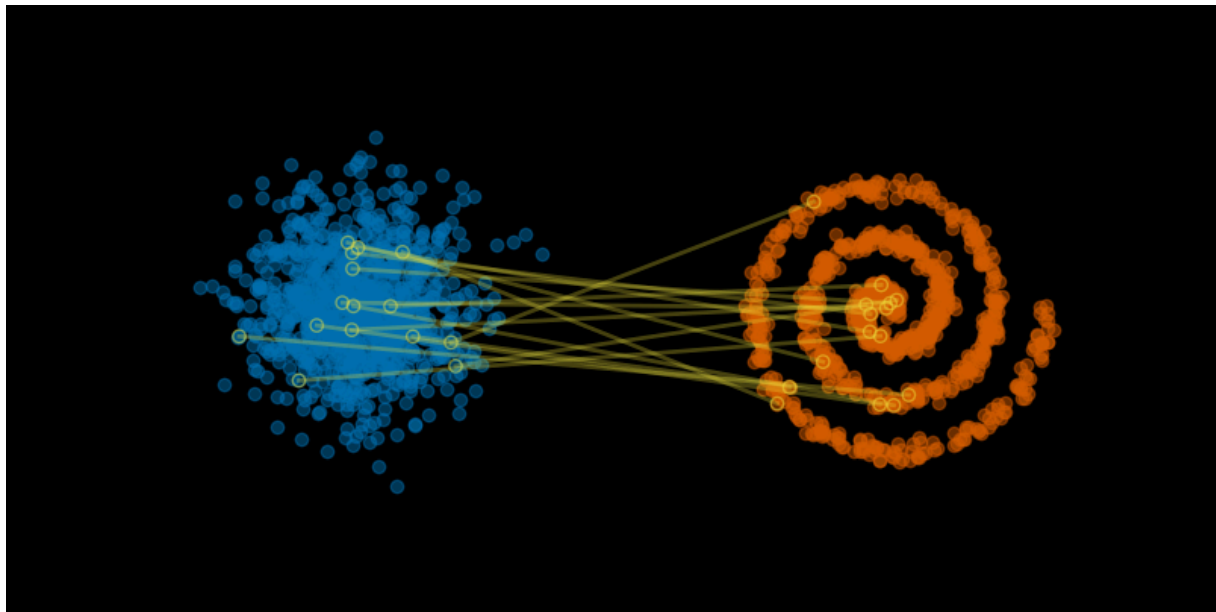


图 4 Randomly pairing source and target points produces a bundle of straight, heavily crossing trajectories. 把源点与目标点随机配对后，得到大量彼此交叉的直线轨迹。

Straight Trajectories == Linear Interpolation 直线轨迹 ⇔ 线性插值

The idea of trying straight trajectories at constant speed is *identical* to simple linear interpolation between source and target data points.¹

尝试以恒定速度沿直线运动，这与在源、目标数据点之间做简单的线性插值是完全等价的。¹

注 1 This is not the only choice for trial trajectories. We could instead use a variance-preserving "cosine interpolation" scheme. This results in noticeably straighter learned flow trajectories than those obtained from linear interpolation, but the linear case is simpler to explain here. See [the executable version of this post] (https://colab.research.google.com/drive/1LFGfvykgZOi9fcdue_Zkt1QYJxPKNtx?usp=sharing) for the option to use cosine interpolation.

这并不是唯一可选的试探轨迹。我们也可以采用保方差的「余弦插值」方案，它所得到的学得流轨迹会明显比线性插值更直；不过线性情形在这里更便于解释。参见本文的可执行版本，其中提供了使用余弦插值的选项。

There are big issues with doing this: The random pairing results in lots of trajectories that cross each other. But this is a *starting point* for Flow Matching.¹

这样做有严重问题：随机配对会产生大量彼此交叉的轨迹。但这是流匹配的一个起点。¹

注 1 "We'll cross the streams." -- Ghostbusters (1984)

「我们要跨越流束了。」——《捉鬼敢死队》(1984)

...well, ok not quite: we're going to allow the *trajectories of individual points* to cross as we train the model. This is a bit "confusing" for the model, which will be trying to learn a velocity field, and that isn't defined where trajectories cross. Eventually, however, the model will learn to estimate the *aggregated motion* of many particles, which will sort of average out to arrive at the "bulk motion" of the flow. This is similar to how the Brownian

motion [8] of many air or water particles averages out on the macroscopic level, giving us streamlines that don't cross.¹

.....好吧，也不完全是这样：训练模型时，我们确实允许单个点的轨迹彼此交叉。这对模型来说确实有点「令人困惑」，因为它要学的是一个速度场，而速度场在轨迹相交处是没有定义的。不过最终，模型会学会估计大量粒子的聚合运动，这种运动在某种意义上会「平均掉」个体差异，得到流的「整体运动（bulk motion）」。这类似于大量空气或水分子做布朗运动时，在宏观尺度上平均出不相交的流线。¹

注 1 A related example from science fiction: in the fictitious "psychohistory" theory of Isaac Asimov's *Foundation* series, the choices and "trajectories" of individual people can't be predicted, but the aggregated development of an entire society follows predictable paths.

科幻中的一个类比：在阿西莫夫《基地》系列虚构的「心理史学」理论中，单个人的选择与「轨迹」无法预测，但整个社会的聚合发展却遵循可预测的路径。

This is why flow matching is about transforming *distributions*, not individual points. The learned velocity field might not exactly match any of our training trajectories, but it captures the statistical flow needed to transform one distribution into another.

这正是为什么流匹配处理的是分布的变换，而非单个点。学到的速度场未必精确匹配任何一条训练轨迹，但它捕捉到了把一个分布变换为另一个分布所需的、统计意义上的流。

Here's a visualization from the code we'll execute later in the lesson. We'll plot...

下面是我们在本课后半段要执行的代码所给出的一幅可视化。我们要画出.....

1. Our naive training trajectories (with crossings)
 2. The actual learned flow field (i.e., the velocity vector field)
 3. The paths (aka "trajectories") that data points follow when flowing with the learned field
1. 我们朴素的训练轨迹（含大量交叉）
 2. 实际学到的流场（即速度矢量场）
 3. 数据点随学到的流场运动时所走的路径（也就是「轨迹」）

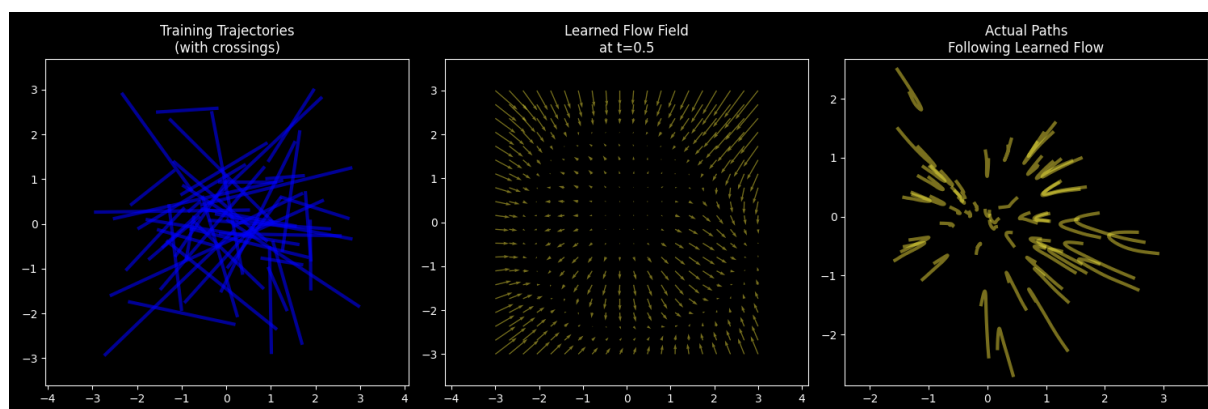


图 5 Produced by the code above; the meaning of the three panels is explained in the text that follows.
由上面的代码生成：三栏的含义见紧接其后的正文说明。

Left: Training data uses simple straight lines (with many crossings). Middle: The learned flow (velocity vector) field is smooth and continuous. Right: Actual trajectories following the flow field don't cross.

左：训练数据使用的是简单直线（交叉很多）。中：学到的流（速度矢量）场是光滑且连续的。右：沿流场运动的实际轨迹不再相交。

2.2 How Are Flows “Learned”?

流是如何被「学习」的？

The goal of the machine learning system is as follows: for any point in space and any time t between 0 and 1, we want to learn the correct *velocity* (direction and speed) that point should move. It's like learning the “wind map” that will blow the starting distribution cloud into the shape of the target distribution cloud.

机器学习系统的目标是：对于空间中的任意一点以及 0 到 1 之间的任意时刻 t ，学会该点应当具有的正确速度（方向与大小）。这就像是在学一张「风场图」，用它把起始分布这团云吹成目标分布的形状。

Since neural networks are such useful engines for approximation and interpolation, we'll let a neural network “learn” to estimate the mapping between locations and times (as inputs), and velocities (as outputs).

既然神经网络是如此好用的逼近与插值引擎，我们就让一个神经网络去「学习」从（位置、时间）到速度的映射。

Terminology: “Simulation Free” 术语说明：「无模拟 (Simulation Free)」

You'll sometimes see flow-matching models being referred to as “simulation free.” This is just an indication that the flow we arrive at is not the result of any explicit simulation of any process (physical or otherwise). The flow obtained arises simply from the aggregation (or “averaging out”) of many particles moving along imagined straight lines and crossing paths.

你有时会看到流匹配模型被称为「无模拟的 (simulation free)」。这只是说明：我们最终得到的流并非对任何过程（物理的或其他）做显式模拟的结果。它仅仅来自大量粒子沿想象出的直线运动、彼此交叉路径的聚合（即「平均」）。

2.2.1 The Neural Network's Job 神经网络的任务

The neural network has one job: given a position in space and a time, to output a velocity vector. *That's all it does.* Below is the code for this model that will “learn” to estimate velocity vectors.

神经网络只有一件事要做：给定空间中的位置与时间，输出一个速度矢量。仅此而已。下面就是这个将「学会」估计速度矢量的模型的代码。

[代码 5: snippet] → 见附录 A

...That's it! Looks pretty simple, right? That's because to make the system work we'll need more than just the velocity field model.

.....就这些！看起来很简单，对吧？这是因为要让整个系统跑起来，我们需要的远不止速度场模型。

Apart from the velocity model (i.e., the neural network, for us), the rest of the software system then uses these generated velocities to move points around. The model's velocities are then used in a differential equation describing the small change to each particle's position r over a short time dt :

除了速度模型（对我们来说就是神经网络）之外，软件系统的其余部分会用生成的速度来移动这些点。模型给出的速度被代入一个微分方程，它描述粒子位置 r 在短时间 dt 内的微小变化：

$$dr = v(r, t) dt$$

That equation is integrated by some (totally separate) numerical integration routine. A popular choice in the machine learning world is the “forward Euler” method, which is simple to implement, but will need to be upgraded (see further below) to get good results.

该方程由某个（完全独立的）数值积分程序来求解。机器学习界常用的选择是「前向欧拉（forward Euler）」法，它实现简单，但要想得到好结果，就需要升级（详见后文）。

[代码 6: Integrator to generate/predict samples using the trained model] → 见附录 A

2.3 Training Code

训练代码

The goal of the training code is twofold:

训练代码的目标有两个：

1. to expose the model to as many locations and times as possible – at least for those times & locations that “matter most”. This exposure is what I’ll refer to as “coverage”.
2. to force the model to learn to generate (approximately) correct velocities at those times and locations.

1. 让模型尽可能多地接触到各种位置与时间——至少是那些「最要紧」的位置与时间。我把这种覆盖称为「coverage（覆盖度）」。
2. 迫使模型学会在这些位置与时间产生（近似）正确的速度。

That’s it. The training code doesn’t actually do any integration or “solving,” but we’ll typically execute the integration on some validation data during training just to visualize “how we’re doing” as the training progresses.

就这些。训练代码本身并不做任何积分或「求解」，但我们通常会在训练过程中对一部分验证数据执行积分，以便可视化地看看训练进行得如何。

[代码 7: viz routine calls integrator, makes pictures] → 见附录 A

输出 / 提示 Output

Testing visualization routines (before training):

训练之前的可视化例程测试：

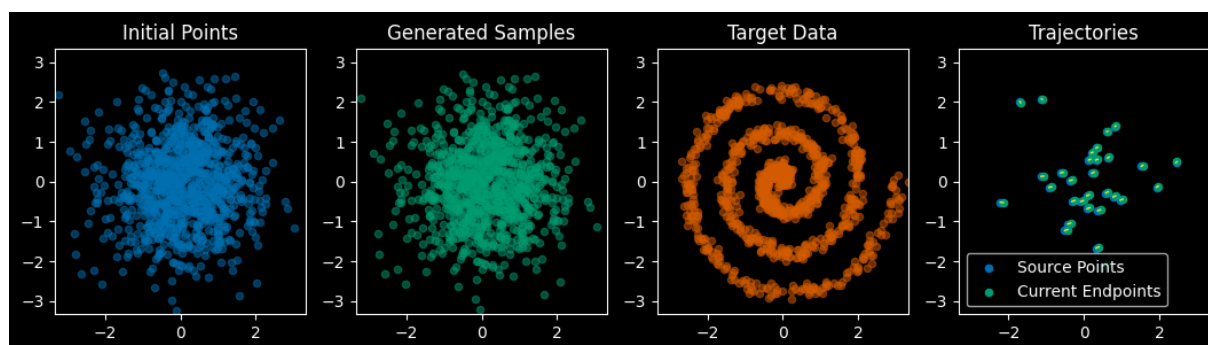


图 6 Output of the visualisation routine: initial points, generated samples, target data and trajectories along the learned flow. 可视化例程的输出：初始点、生成样本、目标数据，以及沿学到的流场运动的轨迹。

The clever part about flow matching is how we train this network. For each training step:

流匹配精妙的地方在于我们如何训练这个网络。每个训练步：

1. Sample random points from our source and target distributions, and pair the points
 2. Sample random times between 0 and 1
 3. Calculate the locations where these points *would* be at those times if they were moving at constant velocity from source to target locations
 4. Calculate what velocity they *would* have at those locations if they were moving at constant velocity
 5. Train the network to *predict* these velocities – which will end up “seeking the mean” when the network has to do the same for many, many points.
1. 从源分布与目标分布中随机采样点，并把它们配对
 2. 在 0 到 1 之间随机采样时间
 3. 计算这些点若从源位置以恒定速度运动到目标位置、在上述时间点应处于的位置
 4. 计算它们在这些位置上若以恒定速度运动、应具有的速度
 5. 训练网络去预测这些速度——当网络要对极大量点做同样的事时，结果会「趋向均值（seeking the mean）」

Some readers may be skeptical: “Could such a scheme even work?” Theoretical assurances to that effect are where the pages probability-math come in. However, machine learning is also an experimental science, as in “Try it and find out!”

有些读者可能会怀疑：「这样的方案真的能行吗？」关于这一点，理论上的保证正是那些概率数学的用武之地。不过，机器学习同时也是一门实验科学，所谓「试一试就知道了！」

Here we run the training code...

下面我们运行训练代码.....

[代码 8: `train_model()` training loop] → 见附录 A

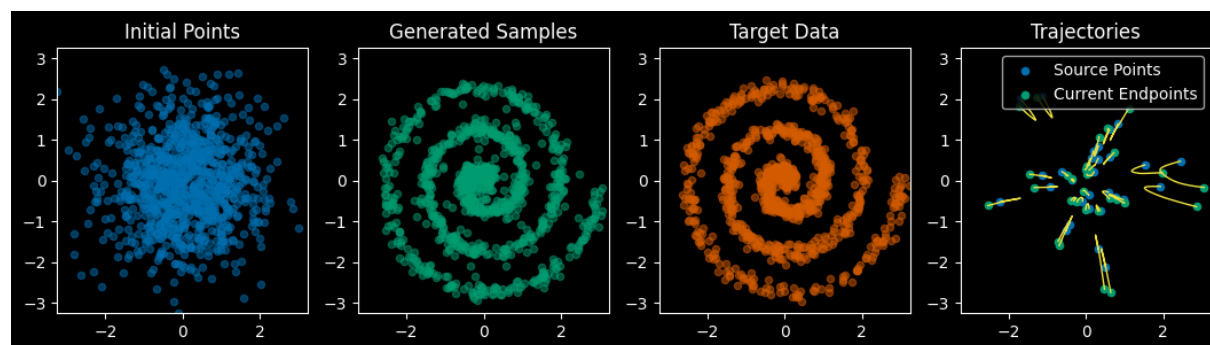


图 7 After training: initial points, generated samples, target data and trajectories.

训练完成后的四联图：初始点、生成样本、目标数据与轨迹。

输出 / 提示 Output

Epoch [100/100], Loss: 1.8315

（训练结束时的输出）

Here’s an animation of integrating points along our model’s flow from start to finish:

下面是让点沿着我们模型的流、从起点积分到终点的动画：

[代码 9: animating points in flow] → 见附录 A

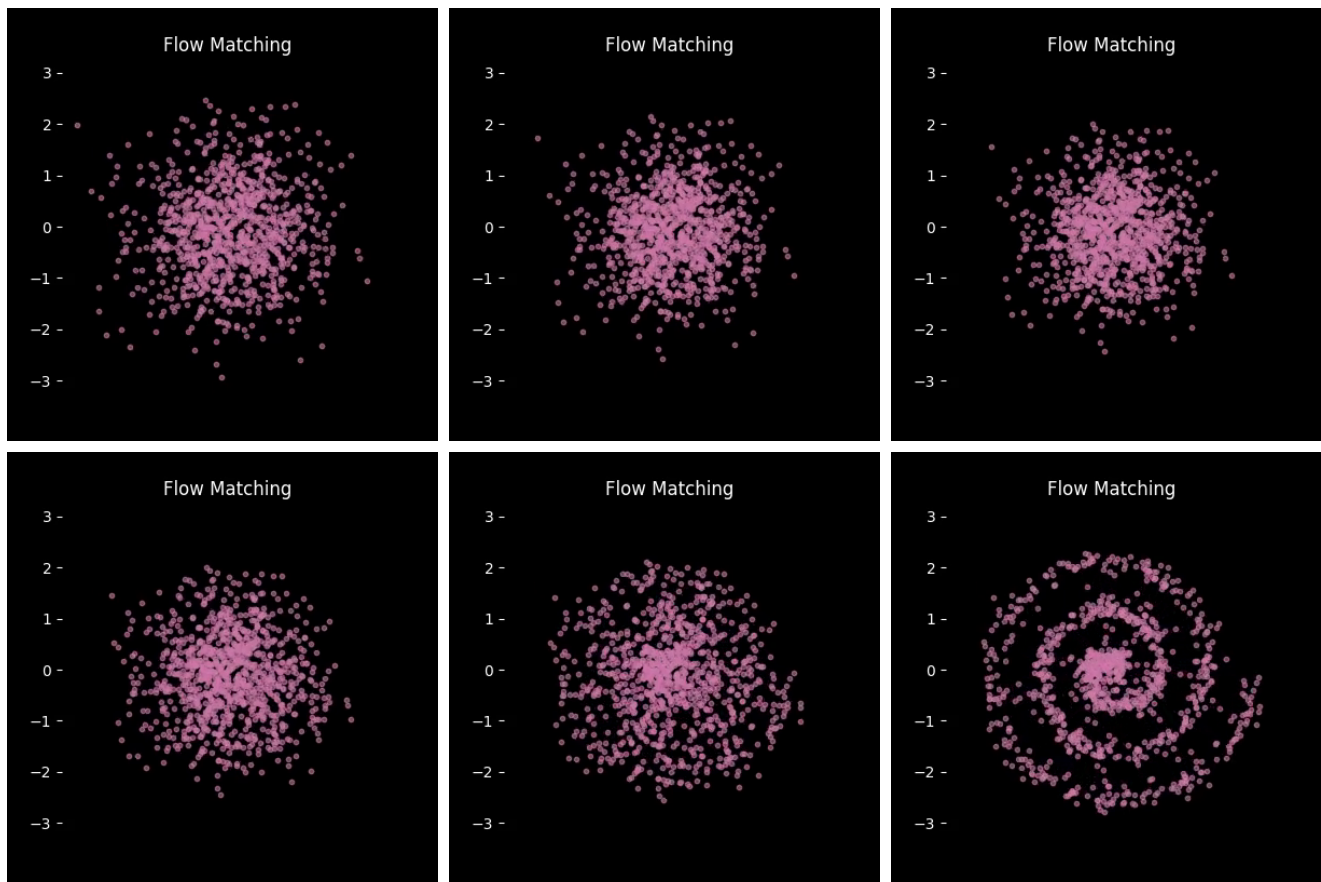


图 8 Points integrated along the learned flow, start to finish. The original is a 50-frame animation; six evenly spaced frames are shown here.

点沿学到的流从起点积分到终点。原文为 50 帧动画，此处按时间等间隔抽取 6 帧。

So, even though we trained using trajectories that crossed, the model learned smooth and non-crossing (but curvy!) trajectories. Here’s a static plot of these:

于是，尽管我们是用相交的轨迹训练的，模型却学到了光滑、不相交（但弯曲！）的轨迹。下面是这些轨迹的静态图：

[代码 10: make the static plot] → 见附录 A

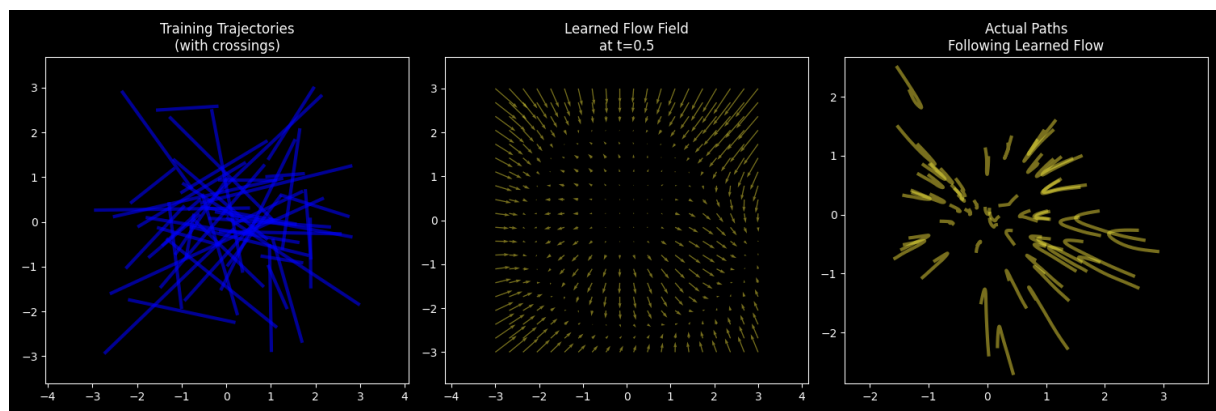


图 9 The same three-panel figure as before, drawn again after training.

与图 5 同一形式的三联图，在训练结束后重新绘制。

Even though the trajectories on the right are smooth and non-crossing, their curviness means that we need to integrate slowly and carefully to avoid accruing significant error. Good news: the “Rectified Flow” paper of Liu et

al ^[2] offers a powerful way to speed up the integration by “straightening” the curved trajectories, a method they call “Reflow.”

尽管右图中的轨迹光滑且不相交，它们的弯曲意味着我们必须缓慢、小心地积分，才能避免累积显著误差。

好消息是：Liu 等人的「Rectified Flow」论文^[2]提供了一种强有力的方法来加速积分——把弯曲的轨迹「拉直」，他们称之为「Reflow」。

3 “Reflow” to Go Straighter & Faster

「Reflow」：让轨迹更直、更快

The Reflow idea is that, instead of randomly pairing source and target points when choosing straight trajectories, we use “simulated target points” by integrating the source points forward using the learned flow model. Then we use those endpoints as the targets and assume linear motion as before.

Reflow 的思路是：在选择直线轨迹时，不再随机配对源与目标点，而是用学到的流模型把源点向前积分，得到「模拟的目标点」；再把这些端点当作目标，像之前一样假设线性运动。

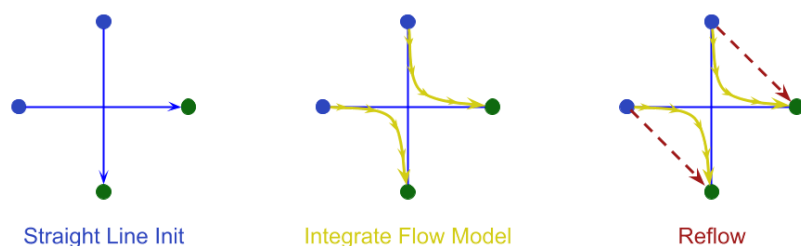


图 10 Schematic of reflow: source points $\rightarrow$ (integrated with the teacher model) $\rightarrow$ simulated target points.

Reflow 的流程示意：源点 $\rightarrow$ (用教师模型积分) $\rightarrow$ 模拟目标点。

This has the effect of straightening out the curved trajectory of the flow matching model, making the new “reflowed” trajectories much easier and faster to integrate!

这样做的效果是拉直匹配模型原本弯曲的轨迹，使新的「reflow 后」的轨迹更容易、也更快速地积分！

Essentially, Reflow is a “teacher-student” paradigm in which the (pre-)trained flow-matching model is the teacher, and the new Reflowed model is the student. One can also think of this as a kind of distillation, akin to “consistency models” [9].

本质上，Reflow 是一种「教师—学生」范式：训练好的（预训练）流匹配模型是教师，新的 Reflow 模型是学生。也可以把它看作一种蒸馏，类似「一致性模型（consistency models）」[9]。

Before we can rely on those integrated endpoints, we should make a couple of improvements to how we use the model we just trained.

在依赖这些积分得到的端点之前，我们应该先对刚训练好的模型的使用方式做几项改进。

3.1 Upgrading Our Gear First

先升级我们的装备

Neither of these upgrades require retraining the velocity model. They just help to make more efficient, accurate use of it while integrating (i.e., while moving data points along the flow) so we’ll have an effective “teacher” for the “student” Reflow model we’ll train below.

这两项改进都不需要重新训练速度模型，它们只是帮助我们在积分时（即沿流推动数据点时）更高效、更精确地使用它，从而为下面要训练的「学生」Reflow 模型提供一个真正有效的「教师」。

3.1.1 More Points Where Needed (via Time Warping) 在需要处加密采样点（时间扭曲）

You may notice that many of the trajectories are sharply curved in the middle but are straight near the start and end. Just as you’d slow down when driving around a sharp turn, we should take smaller integration steps in these curved regions for the sake of accuracy.

你可能会注意到，许多轨迹在中段弯曲得很厉害，而在起点和终点附近却比较直。就像开车过急弯要减速一样，在这些弯曲区域我们应该取更小的积分步长，以保证精度。

The idea of non-uniform temporal sampling appears throughout generative models. Esser et al.'s "FLUX" paper^[3] specifically designs their sampling distribution to concentrate points in the middle of the integration where accuracy is most crucial. The same principle applies here:

非均匀时间采样的思想贯穿于各类生成模型。Esser 等人^[3]的 FLUX 论文就专门设计了采样分布，把采样点集中在积分中段——那里精度最关键。同样的原则在这里也适用：

- during **training**, we want good "coverage" to expose the model to significant places and times where it needs to learn to approximate the correct velocities, and
- during **inference (i.e., integration)**, high-curvature segments will require shorter time steps to maintain accuracy (i.e., to keep from missing the intended destination).
- 在训练阶段，我们希望有良好的「覆盖度」，让模型接触到那些它最需要学会逼近正确速度的关键位置与时间；
- 在推理（即积分）阶段，高曲率区段需要更短的时间步，才能保持精度（即不至于错过本该到达的终点）。

One handy S-shaped time-warping function is this polynomial that lets us vary the concentration of points¹:

一个很方便的 S 形时间扭曲（time warping）函数是下面这个多项式，它让我们可以调节采样点的集中程度：¹

注 1 The $f(t)$ shown here is a close approximation to the "mode function" Eq. 20 in , with their s being about $(1.75 - s_{\text{mine}})$, and with $t \rightarrow 1-t$. The $f(t)$ blue line is right underneath their purple line in the Desmos graph below -- we didn't plan that, just similar minds at work! Both our curves can do the Karras et al cosine schedule, shown in green in the the Desmos figure. 这里的 $f(t)$ 是对某文献中「mode function」式(20)的一个很好的近似，只需把他们的 s 取为约 $1.75 - s_{\text{mine}}$ ，并令 $t \rightarrow 1 - t$ 。在下面的 Desmos 图中，我们的蓝色 $f(t)$ 曲线正好压在他们那条紫色曲线下面——这不是事先商量好的，纯属英雄所见略同！我们的曲线也都能实现 Karras 等人的余弦调度，即 Desmos 图中绿色那条。

$$f(t) = 4(1-s)t^3 + 6(s-1)t^2 + (3-2s)t, \quad t \in [0, 1], \quad s \in [0, 3/2]$$

[原文此处嵌入交互式 Desmos 图: <https://www.desmos.com/calculator/hfjxlwymz?embed>]

[原文此处为可交互的 Desmos 图像，PDF 中无法嵌入；请访问上面的链接。图中对比了时间扭曲函数 $f(t)$ 与 Karras 余弦调度等曲线。]

The parameter s is the slope at $t=1/2$, and controls where points concentrate: values between 0 and 1 give us more points in the middle, which is exactly what we want for these curved trajectories. The value $s=0.5$ is a good choice, as we'll see shortly.

参数 s 是 $t = 1/2$ 处的斜率，控制采样点集中在何处：0 到 1 之间的取值会让中段点更多，这正是我们对这些弯曲轨迹想要的。 $s = 0.5$ 是个不错的选择，稍后就能看到。

This approach can improve accuracy and/or require fewer total integration steps. Let's look at the results of different amounts of time-warping around a simple parabola:

这种做法可以提高精度，并且 / 或者减少所需的总积分步数。下面看看围绕一个简单抛物线、采用不同时间扭曲程度的结果：

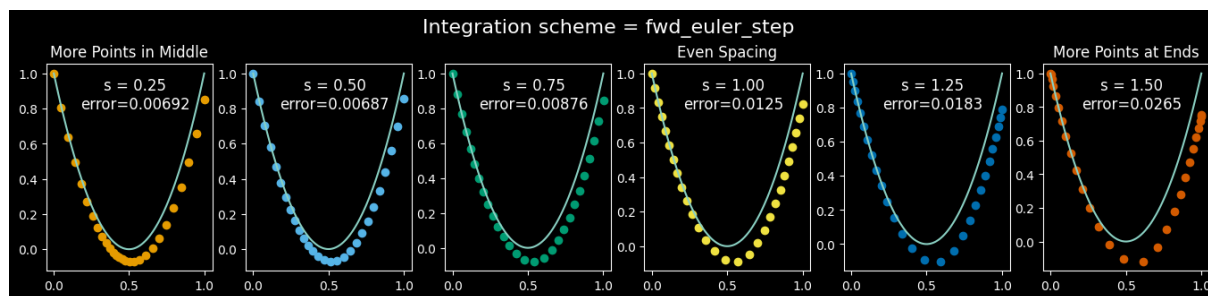


图 11 Forward-Euler integration around a parabola, with different amounts of time warping.

在使用前向欧拉积分的情况下，不同时间扭曲强度围绕抛物线的积分结果。

While the results for $s=0.5$ are better than the others, we see that *none* of these examples make it all the way around the parabola (to the point $(1,1)$)! If we're going to be using the endpoints integrated from the flow matching model as proxies for the true target data, we should have some confidence that those endpoints are actually "reaching" the target data. We could add more (smaller) steps to the integration, but there's another way: upgrade the integration (i.e. sampling) operation to a higher order of accuracy.

虽然 $s = 0.5$ 的结果好于其他取值，但我们看到这些例子没有一个能绕抛物线走完全程（到达点 $(1,1)$ ）！如果我们打算用从流匹配模型积分得到的端点来代理真实目标数据，就该对这些端点确实能「抵达」目标数据有一些信心。我们可以增加更多（更小的）积分步，但还有另一条路：把积分（即采样）操作升级到更高阶的精度。

Note: 注意

Although retraining the flow-matching model with time warping isn't "required," it can improve results by better sampling the domain where accurate velocity approximations matter most. The [executable version of this post](#) offers the option to retrain at this point, but for our toy problem the improvement is minimal.

虽然「并非必须」配合时间扭曲重新训练流匹配模型，但这样做能更好地在「速度逼近最要紧」的区域采样，从而改善结果。本文的可执行版本提供了在此处重新训练的选项，但对于我们这个玩具问题，提升很有限。

3.1.2 Better Integration / Sampling 更好的积分 / 采样格式

Although forward Euler is surprisingly popular in ML circles, those with simulation backgrounds eye it with suspicion: despite being fast (*per step*) and easy to implement, it's also highly inaccurate and can lead to instabilities. The poor accuracy may not be an issue when everything's an approximation anyway, but we can do a lot better.

尽管前向欧拉法在机器学习圈里出奇地流行，有数值模拟背景的人对它却颇存疑虑：它虽然（单步）快、实现简单，但精度很差，还可能引发不稳定。在一切都只是近似的情形下，精度差或许无伤大雅，但我们完全可以做得更好。

People who work with diffusion models know this. For example, Katherine Crowson's k-diffusion package¹ offers a bevy of integration choices. For now, we'll just implement the popular 4th-order Runge-Kutta (RK4) scheme. Per step, it's more "expensive" than forward Euler in that each RK4 step requires 4 function evaluations instead of forward Euler's single evaluation, and RK4 requires some extra storage, but the advantages you gain in accuracy are seriously worth it. Since our learned flow trajectories are *smooth*, a few large steps with a higher-order integrator like RK4 can totally smoke the earlier forward Euler results.

做扩散模型的人对此深有体会。例如 Katherine Crowson 的 k-diffusion 包⁴提供了大量积分方案可选。眼下我们只实现最流行的四阶 Runge–Kutta (RK4) 格式。单步而言，它比前向欧拉法更「贵」：每个 RK4 步需要 4 次函数求值，而非前向欧拉的 1 次，并且 RK4 还需要额外的存储；但它在精度上的收益绝对值得。由于我们学到的流轨迹是光滑的，用 RK4 这类高阶积分器走几个大步，就能彻底碾压之前的前向欧拉结果。

注 1 <https://github.com/crowsonkb/k-diffusion>

<https://github.com/crowsonkb/k-diffusion>

[代码 12: RK4 step] → 见附录 A

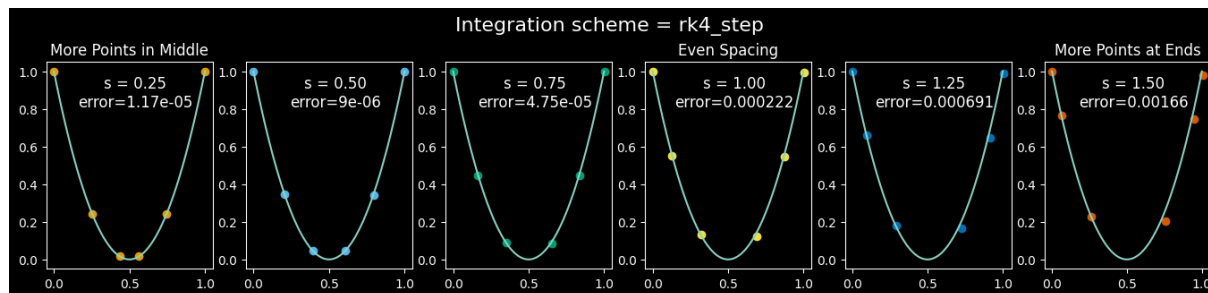


图 12 The same parabola integrated with RK4 instead: far more accurate, and with fewer total function evaluations. 改用 RK4 积分同一抛物线：精度显著提高，且总的函数求值次数更少。

It's cool how the RK4 results, despite showing much less error than the Euler results, actually involve less computational cost in terms of total number of function evaluations, although the RK4 scheme needs 4 times the storage compared to forward Euler. (The good news is that no PyTorch gradients need to be stored; the integrator is only ever used when the model is in “eval” mode.)

有意思的是：RK4 的结果误差远小于欧拉法，但从函数求值总次数来看，其计算代价反而更低——尽管 RK4 所需的存储是前向欧拉的 4 倍。（好消息是，不需要存储 PyTorch 梯度；积分器只在模型处于「eval」模式时使用。）

Note: 注意

We're only going to use the RK4-powered integrator to run inference on the flow matching model to provide the "simulated target data" while we train the Reflow model. Once we have the Reflow model trained, the new trajectories we get from it will be so straight that even forward Euler should be "good enough"!

我们只会在对流匹配模型做推理时使用 RK4 积分器，用来生成训练 Reflow 模型所需的「模拟目标数据」。一旦 Reflow 模型训练完成，它给出的新轨迹会非常直，以至于连前向欧拉法都「足够好了」！

3.2 Learning to “Reflow”

学习 [Reflow]

When we train the “Reflowed” model, aka the student model, note that the “target data” will no longer be supplied by the true target data anymore. Rather, we will be using the trajectory endpoints integrated/generated using the teacher model, i.e. the pretrained flow matching model.

训练「Reflow 后」的模型（也就是学生模型）时要注意：「目标数据」不再由真实目标数据提供，而是使用教师模型（即预训练好的流匹配模型）积分 / 生成的轨迹端点。

So we might ask, “How close of an approximation are those learned flow endpoints to the real thing?” We’re going to be approximating an approximation, but how good is the first approximation?

于是我们可能会问：「这些学到的流端点与真实目标到底有多接近？」我们要逼近的是一个近似——那么这个第一层近似到底有多好？

Let’s take a brief look...

我们简单看一眼.....

[代码 13: code] → 见附录 A

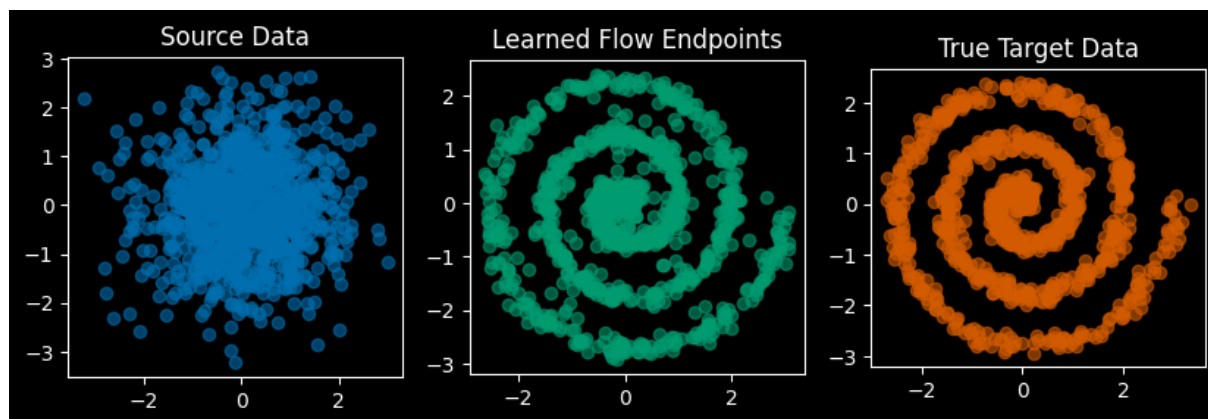


图 13 An endpoint check carried out before training the reflow model.

训练 Reflow 模型之前所做的一次端点检查。

....ok, so we see the learned outputs are a bit different from the true data, but they’re not bad. Let’s now train the “reflow” model.

.....嗯，可以看到学到的输出与真实数据有一点差别，但还不算差。现在我们来训练「reflow」模型。

3.2.1 Train the Reflowed Model 训练 Reflow 模型

There’s one small but crucial change from the previous training code to this one, namely what we use as target data:

与之前的训练代码相比，这里只有一处微小但关键的改动，即我们用什么作为目标数据：

[代码 14: snippet] → 见附录 A

[代码 15: new training loop w/ ReFlowed targets] → 见附录 A

[代码 16: snippet] → 见附录 A

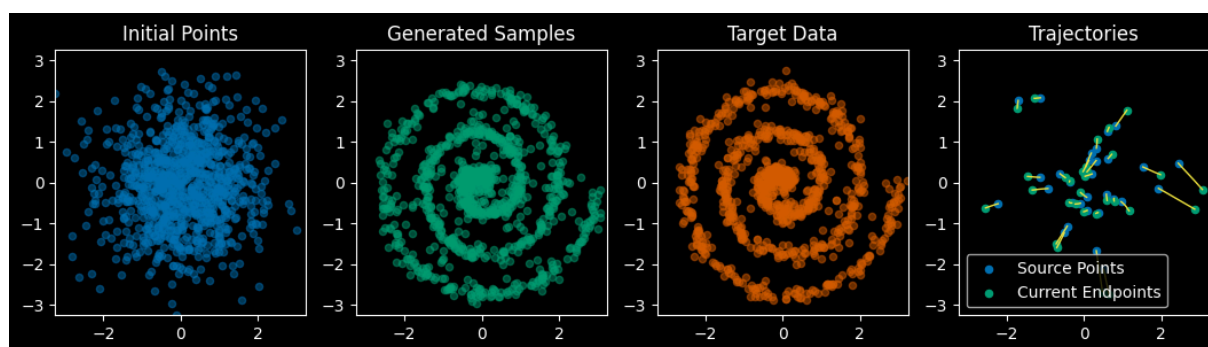


图 14 Inputs, generated endpoints and target trajectories of the reflowed model (four panels).

Reflow 后模型的输入、生成端点与目标轨迹（四个面板）。

Now look how straight the trajectories are! Let's compare animations of the original flow matching model with the "Reflowed" model:

看看现在轨迹有多直！我们把原始流匹配模型与「Reflow 后」模型的动画做个对比：

[代码 17: snippet] → 见附录 A

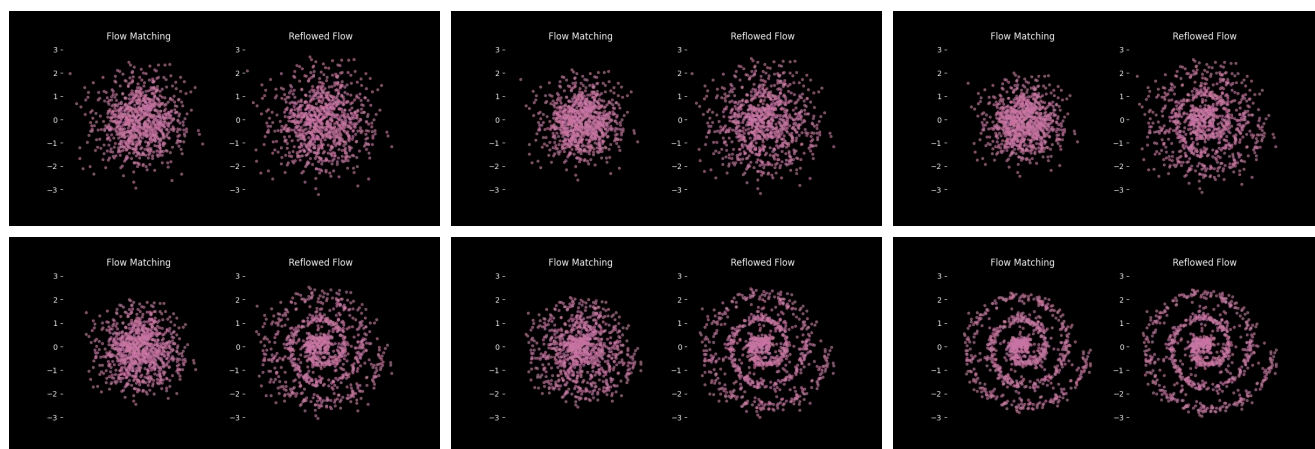


图 15 Left: the original flow-matching model; right: the reflowed model. On the left data first moves inward and back out; on the right it goes straight from start to finish. Six evenly spaced frames of the original animation.

左：原始流匹配模型；右：Reflow 后模型。左图中数据先向内运动再折回，右图则从起点径直走到终点。原动画经等间隔抽取 6 帧。

Notice how the flow matching trajectories on the left have the data moving inward a ways and then back out, whereas the reflowed trajectories move directly from start to finish with no backtracking.

注意左图中流匹配的轨迹会让数据先向内走一段再折回来，而 reflow 后的轨迹则从起点径直走到终点，不再折返。

The next movie shows an animation of "streamlines" with arrows for the local vector field. Note how the shapes on the right change very little over time compared to those on the left. We'll say a bit more about that below.

下一段动画展示的是带局部矢量场箭头的「流线」。注意右图的形状随时间变化非常小，而左图则变化明显。这背后的原因我们后面还会再谈。

[代码 18: streamline animation] → 见附录 A

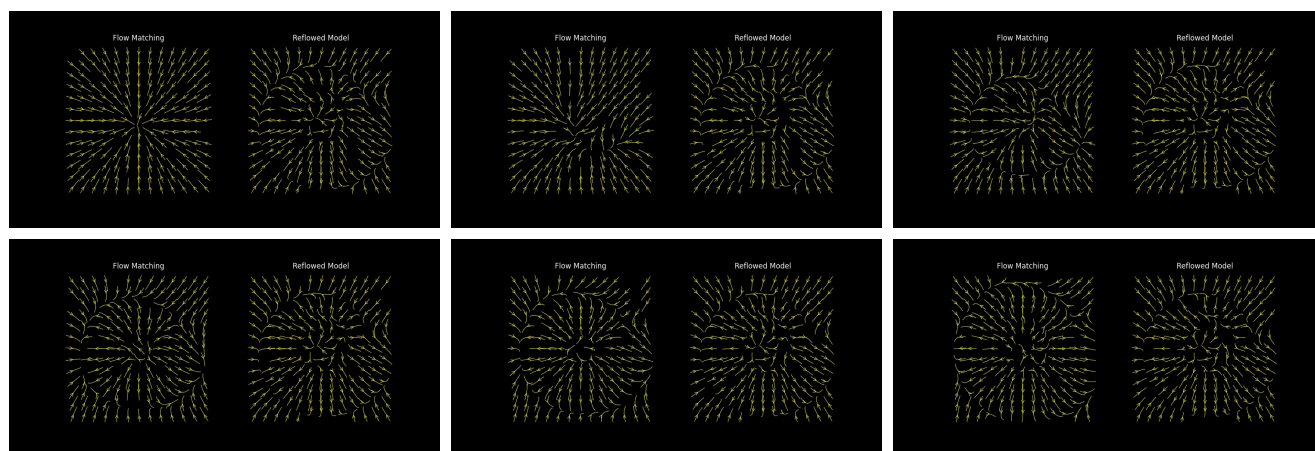


图 16 Streamlines with local vector-field arrows, evolving over time. The shapes on the right (reflowed) change very little compared with those on the left. Six evenly spaced frames of the original animation.

带局部矢量场箭头的流线随时间演化。与左侧相比，右侧（reflow 后）的形状变化非常小。原动画经等间隔抽取 6 帧。

4 Connecting with Other Models

与其他模型的联系

4.1 From Dots to Images, Audio, etc..

从二维点到图像、音频.....

How to move on from 2D dots to things like images, text, audio,...etc? We need only consider that the dimensionality of the velocity model is the same as that of the data itself. Put differently, one can regard the velocity model as supplying a tiny “change” to the data, whatever form that data is in. And the “straight line” trajectory used during training? That’s just linear interpolation between the (initially randomly-paired) source data and the target data. So for images, we will get a “velocity image”, which will tell us how to change the R,G,B values of every pixel in an image. This is where U-Nets and Attention come into play, to efficiently compute the “image-to-image” task of supplying a “velocity image” given an input image distribution (which may just be noise). For audio, regardless of the representation, the velocity model will tell us how to slightly change the component values in that representation. We then just integrate all the little changes as we did with the dots.

如何从二维的点推广到图像、文本、音频等对象？我们只需注意：速度模型的维度与数据本身的维度相同。

换个说法，可以把速度模型看作是在给数据一个微小的「改变」，无论数据是什么形式。那么训练时用的「直线」轨迹呢？那不过是对（最初随机配对的）源数据与目标数据做线性插值。因此对图像而言，我们会得到一个「速度图像」，它告诉我们图像中每个像素的 R、G、B 值应当如何改变。这正是 U-Net 与注意力机制大显身手之处——在给定输入图像分布（可能只是噪声）的情况下，高效地完成「给出速度图像」这一 image-to-image 任务。对音频而言，无论采用什么表示形式，速度模型都会告诉我们该表示中各分量应如何微调；随后我们像处理那些点一样，把所有微小改变积分起来即可。

- The reader may wish to view an MNIST image example such as Tadao Yamaoka’s [\[10\]](#).
- 读者或许想看看 MNIST 图像的示例，例如 Tadao Yamaoka 的 [\[10\]](#)。

4.2 Diffusion Models

扩散模型

Diffusion models, aka “score-based models,” share similarities with flow models in that both learn vector fields that transform points in the sample space. In “standard” diffusion models (DDPMs, [\[11\]](#)), the process is stochastic, constantly injecting fresh noise to simulate Brownian motion [\[8\]](#), unlike the deterministic nature of flow models. However, diffusion models also admit deterministic sampling strategies, notably DDIM (Denoising Diffusion Implicit Models [\[12\]](#)), which follows a “probability flow ODE” without noise injection, making it more directly comparable to flow-matching. The key distinction is that flow-matching allows for more flexible choices of intermediate distributions and trajectories, while still sharing the same fundamental mathematical structure¹. One practical difference is that curved flow-matching trajectories, being smooth, benefit greatly from higher-order integration schemes like RK4, whereas the non-smooth stochastic trajectories of standard diffusion models benefit less from such schemes. There are many more connections between diffusion models and flow-based models, however for brevity’s sake we’ll regard such further discussions as beyond the scope of this tutorial. For greater detail on the connections between diffusion models and flow-based models, we recommend the blog post by Gao et al [\[13\]](#).

扩散模型（又称「基于分数的模型」）与流模型有相似之处：两者都学习能把样本空间中的点进行变换的矢量场。在「标准」扩散模型（DDPM [\[11\]](#)）中，过程是随机的，不断注入新的噪声来模拟布朗运动 [\[8\]](#)，这与

流模型的确定性本质不同。不过，扩散模型也允许确定性采样策略，最著名的是 DDIM（Denoising Diffusion Implicit Models^[12]）：它沿着一条「概率流 ODE」前进而不注入噪声，因而与流匹配更直接可比。关键区别在于，流匹配允许更灵活地选择中间分布与轨迹，同时仍共享相同的数学结构⁴。一个实践上的差异是：弯曲的流匹配轨迹是光滑的，因而从 RK4 这类高阶积分格式中获益很大；而标准扩散模型非光滑的随机轨迹从这类格式中获益较少。扩散模型与基于流的模型之间还有许多联系，但为简洁起见，我们把进一步讨论视为超出本教程范围。若想更深入了解二者的联系，我们推荐 Gao 等人^[13]的博文。

注 1 Technically, the trajectory and the (variance of the) jitter would be the same quantity for diffusion models, and the choice of constant velocity during training is akin to a choice of (constant-total-amplitude) noise schedule. However, diffusion models and flow models are not fully equivalent, and I am a bit hesitant to delineate 1-to-1 correspondences between them.

严格地说，对扩散模型而言，轨迹与扰动（的方差）本是同一个量；训练时选择恒定速度，类似于选择一种（总幅度恒定的）噪声调度。不过扩散模型与流模型并不完全等价，我不太愿意在二者之间划出一一对应的关系。

4.3 Optimal Transport

最优传输

Interesting observation: See how the Reflowed streamlines in the last movie are approximately stationary (i.e., time-independent)? This connects nicely with Optimal Transport theory, where the Benamou-Brenier formulation^[14] (which has a diffusion-esque equation as on objective) shows that optimal mass transport paths follow constant-velocity geodesics in the Wasserstein metric space. This time-independence property emerges naturally when minimizing transport costs in simple metric spaces, as particles following straight paths at constant speeds achieve optimal transport between distributions.

一个有趣的观察：上段动画中 Reflow 后的流线近似是「定常」的（即不随时间变化），对吧？这与最优传输理论联系得很自然：Benamou-Brenier 形式化^[14]（以一个类扩散的方程作为目标）表明，最优质量传输路径是 Wasserstein 度量空间中的常速测地线。在简单度量空间中最小化传输代价时，这一时间无关性会自然浮现——因为沿直线匀速运动的粒子实现了分布之间的最优传输。

4.4 Normalizing Flows

归一化流

Normalizing flows have the property that they preserve overall probability throughout the flow process. While this would seem to be a nice constraint to satisfy, it appears to be unnecessary for “getting the job done” yet it may even limit the expressiveness of the model compared to the kinds of flows we’ve been talking about in this post. Note: Since I’m pushing a “physics perspective,” a similar “conservation” property arises in the phase space flows of Hamiltonian mechanics, namely that they preserve areas and volumes via Liouville’s Theorem^[15]. The connection between normalizing flows and Hamiltonian systems was applied to generative modeling in the “Neural Hamiltonian Flows” paper of ICLR 2020^[16], and continues in a recent NeurIPS 2024 poster-paper^[17]. Despite these fascinating connections, the exciting thing about flow-matching and rectified flows is that they seem to be effective even in the absence of the explicit conservation properties of normalizing flows and their relatives, and can thus be implemented quickly and simply.

归一化流（normalizing flow）的性质是：在整个流动过程中保持总概率不变。这一约束看似很好，但「把事情做成」似乎并不需要它；与本文讨论的这类流相比，它甚至可能限制模型的表达能力。顺带一提：由于我倾向于「物理视角」，哈密顿力学中的相空间流也有一类类似的「守恒」性质，即通过刘维尔定理^[15]保

持面积与体积。归一化流与哈密顿系统之间的联系曾被用于生成建模，见 ICLR 2020 的「Neural Hamiltonian Flows」^[16]，并在最近的 NeurIPS 2024 poster 论文^[17]中延续。尽管这些联系引人入胜，但流匹配与 rectified flow 令人兴奋之处在于：即使没有归一化流及其近亲那种显式的守恒性质，它们依然有效，因此可以快速、简单地实现。

5 Summary

总结

We've seen that flow matching and rectified flow models can be conceptualized and developed using simple ideas from basic physics. This simplicity, coupled with their power and flexibility, has fueled their rise to state-of-the-art performance. While the toy problem presented here is less demanding than applications like high-resolution image synthesis (for which readers should consult relevant literature^[31]), we hope this introduction has developed your intuition for flow-based generative models. This foundation should help you follow the rapid progress in both academic and industry developments, and give you the confidence to expand on these ideas in your own investigations!

我们已经看到，流匹配与 rectified flow 模型可以用基础物理中的简单思想来理解和构建。这种简洁性，加上它们的强大与灵活，推动了它们走向最先进的性能。虽然本文给出的玩具问题比高分辨率图像合成之类的应用要简单得多（后者读者应查阅相关文献），但我们希望这篇介绍能帮助你建立起对基于流的生成模型的直觉。这一基础应有助于你跟上学界与业界的最新进展，并让你有信心在自己的研究中把这些想法进一步拓展！

References

参考文献

1. Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow Matching for Generative Modeling*. ICLR 2023.
<https://openreview.net/forum?id=PqvMRDCJT9t>
2. Liu, X., Gong, C., & Liu, Q. (2023). *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. ICLR 2023.
<https://openreview.net/forum?id=XVjTT1nw5z>
3. Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., Podell, D., Dockhorn, T., English, Z., & Rombach, R. (2024). *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis*. ICML 2024.
<https://openreview.net/forum?id=FPnUhsQJ5B>
4. Lee, S., Lin, Z., & Fanti, G. (2024). *Improving the Training of Rectified Flows*. arXiv:2405.20320.
<https://arxiv.org/abs/2405.20320>
5. Fjelde, T., Mathieu, E., & Dutordoir, V. (2024). *An Introduction to Flow Matching*. MLG Cambridge blog.
<https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html>
6. Huang, J.-B. (2024). *How I Understand Flow Matching*. YouTube.
https://www.youtube.com/watch?v=DDq_pIfHqLs
7. Abraham, T. M. (2024). *Flow matching and rectified flows are the same*. X.com.
<https://x.com/iScienceLuvr/status/1766700945243881889>
8. *Brownian motion* — Wikipedia, The Free Encyclopedia (2024).
https://en.wikipedia.org/wiki/Brownian_motion
9. Song, Y., Dhariwal, P., Chen, M., & Sutskever, I. (2023). *Consistency Models*. arXiv:2303.01469.
<https://arxiv.org/abs/2303.01469>
10. Yamaoka, T. (2024). *Image generation with Rectified Flow Part 2 (learning MNIST using Scratch implementation)*.
<https://tadaoyamaoka.hatenablog.com/entry/2024/09/29/163801>
11. Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS 33, 6840–6851.
https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584afod967f1ab10179ca4b-Paper.pdf
12. Song, J., Meng, C., & Ermon, S. (2021). *Denoising Diffusion Implicit Models*. ICLR 2021.
13. Gao, R., Hoogeboom, E., Heek, J., De Bortoli, V., Murphy, K. P., & Salimans, T. (2024). *Diffusion Meets Flow Matching: Two Sides of the Same Coin*.
<https://diffusionflow.github.io/>
14. Benamou, J.-D., & Brenier, Y. (2000). *A computational fluid mechanics solution to the Monge–Kantorovich mass transfer problem*. Numerische Mathematik, 84, 375–393.
<https://api.semanticscholar.org/CorpusID:1100384>
15. *Liouville's theorem (Hamiltonian)* — Wikipedia, The Free Encyclopedia (2024).
16. Toth, P., Rezende, D. J., Jaegle, A., Racanière, S., Botev, A., & Higgins, I. (2020). *Hamiltonian Generative Networks*. ICLR 2020.
<https://openreview.net/forum?id=HJenn6VFvB>
17. Holderrieth, P., Xu, Y., & Jaakkola, T. (2024). *Hamiltonian Score Matching and Generative Flows*. arXiv:2410.20470.
<https://arxiv.org/abs/2410.20470>

Appendix A — Full Code

附录 A：完整代码

All code cells of the original post, in the order they appear. They are reproduced verbatim; the authoritative, runnable version is the [Colab notebook](#).

原文全部代码单元，按出现顺序排列，均保持原样。可运行的权威版本请见 Colab notebook。

A.1 Installs & imports 安装与导入

```
# Uncomment to install any missing dependencies:
#%pip install torch numpy matplotlib tqdm plotly

import torch
import numpy as np
%matplotlib inline
import matplotlib.pyplot as plt
from IPython.display import HTML, display, clear_output
from tqdm.notebook import tqdm
import plotly.graph_objects as go
```

A.2 snippet 选择源分布与目标分布的形状

```
# Options are: 'Gaussian', 'Square', 'Heart', 'Spiral', 'Two Gaussians', 'Smiley'
source_data_choice = 'Gaussian'
target_data_choice = 'Spiral'
```

A.3 Utilities, styles, functions, generators, visualization 工具函数、配色、数据生成器与可视化

```
# for accessibility: Wong's color palette: cf. https://davidmathlogic.com/colorblind
#wong_black = [0/255, 0/255, 0/255] # #000000
wong_amber = [230/255, 159/255, 0/255] # #E69F00
wong_cyan = [86/255, 180/255, 233/255] # #56B4E9
wong_green = [0/255, 158/255, 115/255] # #009E73
wong_yellow = [240/255, 228/255, 66/255] # #F0E442
wong_navy = [0/255, 114/255, 178/255] # #0072B2
wong_red = [213/255, 94/255, 0/255] # #D55E00
wong_pink = [204/255, 121/255, 167/255] # #CC79A7
wong_cmap = [wong_amber, wong_cyan, wong_green, wong_yellow, wong_navy, wong_red, wong_pink]

source_color = wong_navy
target_color = wong_red
pred_color = wong_green
line_color = wong_yellow
bg_theme = 'dark' # 'black', 'white', 'dark', 'light'
if bg_theme in ['black', 'dark']:
    plt.style.use('dark_background')
else:
    plt.rcParamsdefaults()

# A few different data distributions
def create_gaussian_data(n_points=1000, scale=1.0):
    """Create a 2D Gaussian distribution"""
    return torch.randn(n_points, 2) * scale

def create_square_data(n_points=1000, scale=3.0): # 3 is set by the spread of the gaussian and spiral
    """Create points uniformly distributed in a square"""
    # Generate uniform points in a square
    points = (torch.rand(n_points, 2) * 2 - 1) * scale
    return points

def create_spiral_data(n_points=1000, scale=1):
    """Create a spiral distribution. i like this one more"""
    noise = 0.1*scale
```

```

#theta = torch.linspace(0, 6*np.pi, n_points) # preferred order? no way
theta = 6*np.pi* torch.rand(n_points)
r = theta / (2*np.pi) * scale
x = r * torch.cos(theta) + noise * torch.randn(n_points)
y = r * torch.sin(theta) + noise * torch.randn(n_points)
return torch.stack([x, y], dim=1)

def create_heart_data(n_points=1000, scale=3.0):
    """Create a heart-shaped distribution of points"""
    square_points = create_square_data(n_points, scale=1.0)

    # Calculate the heart-shaped condition for each point
    x, y = square_points[:, 0], square_points[:, 1]
    heart_condition = x**2 + ((5 * (y + 0.25) / 4) - torch.sqrt(torch.abs(x)))**2 <= 1

    # Filter out points that don't satisfy the heart-shaped condition
    heart_points = square_points[heart_condition]

    # If we don't have enough points, generate more
    while len(heart_points) < n_points:
        new_points = create_square_data(n_points - len(heart_points), scale=1)
        x, y = new_points[:, 0], new_points[:, 1]
        new_heart_condition = x**2 + ((5 * (y + 0.25) / 4) - torch.sqrt(torch.abs(x)))**2 <= 1
        new_heart_points = new_points[new_heart_condition]
        heart_points = torch.cat([heart_points, new_heart_points], dim=0)

    heart_points *= scale
    return heart_points[:n_points]

def create_two_gaussians_data(n_points=1000, scale=1.0, shift=2.5):
    """Create a 2D Gaussian distribution"""
    g = torch.randn(n_points, 2) * scale
    g[:n_points//2, 0] -= shift
    g[n_points//2:, 0] += shift
    indices = torch.randperm(n_points)
    return g[indices]

def create_smiley_data(n_points=1000, scale=2.5):
    "make a smiley face"
    points = []
    # Face circle
    #angles = 2 * np.pi * torch.rand(n_points//2+20)
    #r = scale + (scale/10)*torch.sqrt(torch.rand(n_points//2+20))
    #points.append(torch.stack([r * torch.cos(angles), r * torch.sin(angles)], dim=1))

    # Eyes (small circles at fixed positions)
    for eye_pos in [[-1, 0.9], [1, 0.9]]:
        eye = torch.randn(n_points//3+20, 2) * 0.2 + torch.tensor(eye_pos) * scale * 0.4
        points.append(eye)

    # Smile (arc in polar coordinates)
    theta = -np.pi/6 - 2*np.pi/3*torch.rand(n_points//3+20)
    r_smile = scale * 0.6 + (scale/4)* torch.rand_like(theta)
    points.append(torch.stack([r_smile * torch.cos(theta), r_smile * torch.sin(theta)], dim=1))

    points = torch.cat(points, dim=0) # concatenate first
    points = points[torch.randperm(points.shape[0])] # then shuffle
    return points[:n_points,:]

# Initialize generator functions
source_gen_fn = None
target_gen_fn = None

# Assign generator functions based on user choices
for gen_choice, gen_fn_name in zip([source_data_choice, target_data_choice], ['source_gen_fn', 'target_gen_fn']):
    gen_choice = gen_choice.lower()
    if 'two gaussians' in gen_choice:
        gen_fn = create_two_gaussians_data
    elif 'heart' in gen_choice:
        gen_fn = create_heart_data
    elif 'spiral' in gen_choice:
        gen_fn = create_spiral_data
    elif 'square' in gen_choice:
        gen_fn = create_square_data
    elif 'smiley' in gen_choice:

```

```

        gen_fn = create_smiley_data
    else:
        gen_fn = create_gaussian_data

    if gen_fn_name == 'source_gen_fn':
        source_gen_fn = gen_fn
    else:
        target_gen_fn = gen_fn

# A couple aliases so we can easily switch distributions without affecting later code
def create_source_data(n_points=1000, hshift=0): # hshift can make it a bit easier to see trajectories later
    g = source_gen_fn(n_points=n_points)
    if hshift != 0: g[:,0] += hshift
    return g

def create_target_data(n_points=1000, hshift=0):
    g = target_gen_fn(n_points=n_points)
    if hshift != 0: g[:,0] += hshift
    return g

def plot_distributions(dist1, dist2, title1="Distribution 1", title2="Distribution 2", alpha=0.8):
    """Plot two distributions side by side"""
    plt.close('all')
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    ax1.scatter(dist1[:, 0], dist1[:, 1], alpha=alpha, s=10, color=source_color)
    ax2.scatter(dist2[:, 0], dist2[:, 1], alpha=alpha, s=10, color=target_color)

    ax1.set_title(title1)
    ax2.set_title(title2)

    # Set same scale for both plots
    max_range = max(
        abs(dist1).max().item(),
        abs(dist2).max().item()
    )
    for ax in [ax1, ax2]:
        ax.set_xlim(-max_range, max_range)
        ax.set_ylim(-max_range, max_range)
        ax.set_aspect('equal')

    plt.tight_layout()
    plt.show() # Explicitly show the plot
    plt.close()

def interpolate_color(t, start='blue', end='red'):
    """Interpolate from matplotlib's default green (t=0) to red (t=1)"""
    start_color = plt.cm.colors.to_rgb(start)
    end_color = plt.cm.colors.to_rgb(end)
    return (1-t) * np.array(start_color) + t * np.array(end_color)

def show_flow_sequence(start_dist, end_dist, n_steps=5, c_start=source_color, c_end=target_color):
    """Show the flow as a sequence of static plots"""
    fig, axes = plt.subplots(1, n_steps, figsize=(4*n_steps, 4))

    max_range = max(
        abs(start_dist).max().item(),
        abs(end_dist).max().item()
    )

    for i, ax in enumerate(axes):
        t = i / (n_steps - 1)
        current = start_dist * (1-t) + end_dist * t

        color = interpolate_color(t, start=c_start, end=c_end)
        ax.scatter(current[:, 0], current[:, 1],
                  alpha=0.8, s=10,
                  c=[color])

        ax.set_xlim(-max_range, max_range)
        ax.set_ylim(-max_range, max_range)
        ax.set_aspect('equal')
        ax.set_title(f't = {t:.2f}')

```

```

plt.tight_layout()
plt.show()
plt.close()

# Create our distributions and look at them
n_points = 1000
source, target = create_source_data(n_points), create_target_data(n_points)

plot_distributions(source, target, "Starting Distribution", "Target Distribution")

```

A.4 Make 2D flow-matching diagram with crossing lines 绘制含交叉线的二维流匹配示意图

```

source_L = source.clone()
shift = 5
source_L[:,0] -= shift
target_R = target.clone()
target_R[:,0] += shift # Note: fixed the indexing here from [:0] to[:,0]

fig, ax = plt.subplots(figsize=(8,4))
# show the whole distribution
ax.scatter(source_L[:,0], source_L[:,1], color=source_color, alpha=0.5)
ax.scatter(target_R[:,0], target_R[:,1], color=target_color, alpha=0.5)

# Draw lines connecting points, with source & target points outlined
n_lines = 15
ax.scatter(source_L[:n_lines,0], source_L[:n_lines,1], color=source_color, alpha=0.5,
           facecolor='none', edgecolor=line_color,)
ax.scatter(target_R[:n_lines,0], target_R[:n_lines,1], color=target_color, alpha=0.5,
           facecolor='none', edgecolor=line_color,)
for i in range(n_lines):
    ax.plot([source_L[i,0], target_R[i,0]],
            [source_L[i,1], target_R[i,1]],
            '-', alpha=0.3, color=line_color+ [.9],
            linewidth=2) # or lw=2

ax.set_aspect('equal')
ax.set_xticks([])
ax.set_yticks([])
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['bottom'].set_visible(False)
ax.spines['left'].set_visible(False)
for [x, label] in zip([-shift,shift], ['Source','Target']):
    ax.text(x, 4, label, fontsize=12, color='black', ha='center', va='center',)
plt.show()
plt.close()

```

A.5 snippet 速度场网络 VelocityNet

```

import torch.nn as nn
import torch.nn.functional as F

class VelocityNet(nn.Module):
    def __init__(self, input_dim, h_dim=64):
        super().__init__()
        self.fc_in = nn.Linear(input_dim + 1, h_dim)
        self.fc2 = nn.Linear(h_dim, h_dim)
        self.fc3 = nn.Linear(h_dim, h_dim)
        self.fc_out = nn.Linear(h_dim, input_dim)

    def forward(self, x, t, act=F.gelu):
        t = t.expand(x.size(0), 1) # Ensure t has the correct dimensions
        x = torch.cat([x, t], dim=1)
        x = act(self.fc_in(x))
        x = act(self.fc2(x))
        x = act(self.fc3(x))
        return self.fc_out(x)

# Instantiate the model

```

```
input_dim = 2
model = VelocityNet(input_dim)
```

A.6 Integrator to generate/predict samples using the trained model 用训练好的模型生成 / 预测样本的积分器

```
@torch.no_grad()
def fwd_euler_step(model, current_points, current_t, dt):
    velocity = model(current_points, current_t)
    return current_points + velocity * dt

@torch.no_grad()
def integrate_path(model, initial_points, step_fn=fwd_euler_step, n_steps=100,
                  save_trajectories=False, warp_fn=None):
    """this 'sampling' routine is primarily used for visualization."""
    device = next(model.parameters()).device
    current_points = initial_points.clone()
    ts = torch.linspace(0,1,n_steps).to(device)
    if warp_fn: ts = warp_fn(ts)
    if save_trajectories: trajectories = [current_points]
    for i in range(len(ts)-1):
        current_points = step_fn(model, current_points, ts[i], ts[i+1]-ts[i])
        if save_trajectories: trajectories.append(current_points)
    if save_trajectories: return current_points, torch.stack(trajectories).cpu()
    return current_points

generate_samples = integrate_path # just lil' alias for the probability / diffusion model crowd ;-)
```

A.7 viz routine calls integrator, makes pictures 可视化例程：调用积分器并出图

```
#| code-fold: true
#| code-summary: "Viz code: calls integrator to calc motion given v field, makes pictures"

def viz(val_points, target_samples, trained_model, size=20, alpha=0.5, n_steps=100, warp_fn=None,):
    # Generate and visualize new samples
    device = next(trained_model.parameters()).device
    generated_samples, trajectories = integrate_path(trained_model, val_points.to(device), n_steps=n_steps,
    warp_fn=warp_fn, save_trajectories=True)

    n_viz = min(30, len(trajectories[0])) # Number of trajectories to visualize

    fig, ax = plt.subplots(1,4, figsize=(13,3))
    data_list = [val_points.cpu(), generated_samples.cpu(), target_samples.cpu()]
    label_list = ['Initial Points', 'Generated Samples', 'Target Data', 'Trajectories']
    color_list = [source_color, pred_color, target_color]
    global_max = max( torch.max(torch.abs(torch.cat(data_list))),0)[0][0], torch.max(torch.abs(torch.cat(data_list))),0)
    [0][1] )
    for i in range(len(label_list)):
        ax[i].set_title(label_list[i])
        ax[i].set_xlim([-global_max, global_max])
        ax[i].set_ylim([-global_max, global_max])
        if i < 3: # non-trajectory plots
            ax[i].scatter( data_list[i][:, 0], data_list[i][:, 1], s=size, alpha=alpha,
                           label=label_list[i], color=color_list[i])
        else:
            # Plot trajectory paths first
            for j in range(n_viz):
                path = trajectories[:, j]
                ax[3].plot(path[:, 0], path[:, 1], '-', color=line_color, alpha=1, linewidth=1)

            # Then plot start and end points for the SAME trajectories
            start_points = trajectories[0, :n_viz]
            end_points = trajectories[-1, :n_viz]
            ax[3].scatter(start_points[:, 0], start_points[:, 1], color=source_color, s=size, alpha=1, label='Source
Points')
            ax[3].scatter(end_points[:, 0], end_points[:, 1], color=pred_color, s=size, alpha=1, label='Current
Endpoints')
            ax[3].legend()

    plt.show()
    plt.close()
```

```
# Visualize the data
n_samples = 1000
source_samples = create_source_data(n_samples)
target_samples = create_target_data(n_samples)
val_points = create_source_data(n_samples)
print("Testing visualization routines (before training):")
viz(val_points, target_samples, model)
```

A.8 train_model() training loop train_model() 训练循环

```
#| code-fold: true
#| code-summary: "Code for train_model() training loop"
import torch.optim as optim

def train_model(model, n_epochs=100, lr=0.003, batch_size=2048, status_every=1, viz_every=1, warp_fn=None):
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()
    step, n_steps = 0, 100
    device = next(model.parameters()).device

    for epoch in range(n_epochs):
        model.train()
        pbar = tqdm(range(n_steps), leave=False)
        for _ in pbar:
            step += 1
            optimizer.zero_grad()

            # by randomly generating new data each step, we prevent the model from merely memorizing
            source_samples = create_source_data(batch_size).to(device)
            target_samples = create_target_data(batch_size).to(device)

            t = torch.rand(source_samples.size(0), 1).to(device) # random times for traning
            if warp_fn: t = warp_fn(t) # time warp is good for coverage but not as helpful for training as it is
            during integration/sampling
            interpolated_samples = source_samples * (1 - t) + target_samples * t
            line_directions = target_samples - source_samples

            drift = model(interpolated_samples, t)
            loss = loss_fn(drift, line_directions)

            loss.backward()
            optimizer.step()

            status_str = f'Epoch [{epoch + 1}/{n_epochs}], Loss: {loss.item():.4f}'
            pbar.set_description(status_str)

            if (epoch + 1) % viz_every == 0:
                model.eval()
                clear_output(wait=True) # Clear previous plots
                viz(val_points, target_samples[:val_points.shape[0]], model)
                plt.show()
                plt.close() # Close the figure to free memory
                model.train()

            if epoch==n_epochs-1: print(status_str) # keep last status from being cleared

    return model
```

A.9 animating points in flow 让点沿流运动的动画

```
#| code-fold: true
#| code-summary: "Code for animating points in flow"
import matplotlib.animation as animation
from IPython.display import HTML, display, clear_output
from matplotlib import rc
import os

@torch.no_grad()
def create_flow_animation(start_dist, models, titles=None, figsize=None, n_frames=50,
                          step_fn=fwd_euler_step, n_steps=100, warp_fn=None, save_file=None, height=4):
```

```

"""
Create an animation showing multiple distribution flows

Args:
    start_dist: Starting distribution
    models: List of models to animate
    titles: List of titles for each subplot (optional)
    figsize: Figure size (optional)
    n_frames: Number of animation frames
    integrator: Integration function to use
    jitter: Amount of jitter to add
    save_file: Path to save animation (optional)
    height: Height of each subplot
"""
plt.close('all') # Close all open figures

if not isinstance(models, list): models = [models]
n_plots = len(models)

if titles is None:
    titles = [f'Flow {i+1}' for i in range(n_plots)]
elif len(titles) != n_plots:
    raise ValueError(f"Number of titles ({len(titles)}) must match number of models ({n_plots})")

# Calculate figure size
if figsize is None:
    figsize = [height * n_plots, height]

# Create subplots
fig, axes = plt.subplots(1, n_plots, figsize=figsize)
if n_plots == 1:
    axes = [axes]
plt.close() # Close the figure immediately

# Initialize scatters and trajectories
scatters = []
all_trajectories = []

# Generate trajectories for each model
max_range = abs(start_dist).max().item()

for i, model in enumerate(models):
    device = next(model.parameters()).device
    end_dist, trajectories = integrate_path(model, start_dist.clone().to(device), n_steps=n_frames,
                                           step_fn=step_fn, warp_fn=warp_fn, save_trajectories=True)
    all_trajectories.append(trajectories.cpu())
    scatters.append(axes[i].scatter([], [], alpha=0.6, s=10, color=wong_pink))

    # Update max range
    max_range = max(max_range, abs(end_dist.cpu()).max().item())

# Set up axes
for i, ax in enumerate(axes):
    ax.set_xlim((-max_range, max_range))
    ax.set_ylim((-max_range, max_range))
    ax.set_aspect('equal')
    ax.set_xticks([])
    for spine in ['top', 'right', 'bottom', 'left']:
        ax.spines[spine].set_visible(False)
    ax.set_title(titles[i])

def init():
    """Initialize animation"""
    for scatter in scatters:
        scatter.set_offsets(np.c_[[], []])
    return tuple(scatters)

def animate(frame):
    """Update animation frame"""
    # Update axis limits (in case they need to be adjusted)
    for ax in axes:
        ax.set_xlim((-max_range, max_range))
        ax.set_ylim((-max_range, max_range))

    # Update scatter positions
    for scatter, trajectories in zip(scatters, all_trajectories):

```

```

        scatter.set_offsets(trjectories[frame].numpy())

    return tuple(scatters)

# Create animation
anim = animation.FuncAnimation(fig, animate, init_func=init,
                              frames=n_frames, interval=20, blit=True)

# Handle saving or displaying
if save_file:
    os.makedirs(os.path.dirname(save_file), exist_ok=True)
    anim.save(save_file, writer='ffmpeg', fps=30)
    return HTML(f"""<center><video height="350" controls loop><source src="{anim_file}" type="video/mp4">
        Your browser does not support the video tag. </video></center>""")
else: # direct matplotlib anim offers better controls but makes ipynb file size huge
    rc('animation', html='jshtml')
    return HTML(anim.to_jshtml())
plt.close()

anim_file = 'images/particles_fm.mp4'
create_flow_animation(val_points.clone(), models=[fm_model], titles=['Flow Matching'],
                    n_frames=50, save_file=anim_file)

```

A.10 make the static plot 绘制静态对比图

```

@torch.no_grad()
def plot_training_trajectories_vs_learned_flow(model):
    """Compare training trajectories with learned flow field"""
    plt.figure(figsize=(15, 5))

    # 1. Plot some training trajectories
    plt.subplot(131)
    n_trajs = 50 # Number of trajectories to show
    device = next(model.parameters()).device
    source = create_gaussian_data(n_trajs)
    target = create_square_data(n_trajs)
    current_points = source.clone().to(device)

    # Plot straight-line trajectories from source to target
    times = torch.linspace(0, 1, 20)
    for i in range(n_trajs):
        traj = source[i:i+1] * (1 - times.reshape(-1, 1)) + target[i:i+1] * times.reshape(-1, 1)
        plt.plot(traj[:, 0], traj[:, 1], 'b-', alpha=0.6, linewidth=3)
    plt.title('Training Trajectories\n(with crossings)')
    plt.axis('equal')

    # 2. Plot learned flow field
    plt.subplot(132)
    x = torch.linspace(-3, 3, 20)
    y = torch.linspace(-3, 3, 20)
    X, Y = torch.meshgrid(x, y, indexing='ij')
    points = torch.stack([X.flatten(), Y.flatten()], dim=1).to(device)

    # with torch.no_grad():
    #     t = 0.5 # Show flow field at t=0.5
    #     ones = torch.ones(points.size(0), 1)

    ones = torch.ones(points.size(0), 1).to(device)
    t = ones * (0.5)
    velocities = model(points, t).cpu()
    #print("points.shape, ones.shape = ", points.shape, ones.shape)
    #velocities = model(points, t*ones)
    points = points.cpu()
    plt.quiver(points[:, 0], points[:, 1],
              velocities[:, 0], velocities[:, 1],
              alpha=0.5, color=line_color, linewidth=3)
    plt.title('Learned Flow Field\nat t=0.5')
    plt.axis('equal')

    # 3. Plot actual paths taken using learned flow
    plt.subplot(133)

```

```

source = create_gaussian_data(n_trajs)

# Use RK4 to follow the learned flow
paths = []
n_steps = 20
dt = 1.0 / n_steps

with torch.no_grad():
    ones = torch.ones(current_points.size(0), 1).to(device)
    for i in range(n_steps):
        paths.append(current_points.clone())

        # RK4 step
        t = ones * (i * dt)
        k1 = model(current_points, t)
        k2 = model(current_points + k1 * dt/2, t + dt/2)
        k3 = model(current_points + k2 * dt/2, t + dt/2)
        k4 = model(current_points + k3 * dt, t + dt)

        current_points = current_points + (k1 + 2*k2 + 2*k3 + k4) * dt/6

paths = torch.stack(paths).cpu()

# Plot the actual paths
for i in range(n_trajs):
    traj = paths[:, i, :]
    plt.plot(traj[:, 0], traj[:, 1], color=line_color, alpha=0.5, linewidth=3)
plt.title('Actual Paths\nFollowing Learned Flow')
plt.axis('equal')

plt.tight_layout()
plt.savefig('images/cross_uncross_plot.png')
plt.show()
plt.close()

# Run the visualization
plot_training_trajectories_vs_learned_flow(fm_model)

```

A.11 warp_time() and parabola example warp_time() 与抛物线示例

```

def warp_time(t, dt=None, s=.5):
    """Parametric Time Warping: s = slope in the middle.
       s=1 is linear time, s < 1 goes slower near the middle, s>1 goes slower near the ends
       s = 1.5 gets very close to the "cosine schedule", i.e. (1-cos(pi*t))/2, i.e. sin^2(pi/2*x)"""
    if s<0 or s>1.5: raise ValueError(f"s={s} is out of bounds.")
    tw = 4*(1-s)*t**3 + 6*(s-1)*t**2 + (3-2*s)*t
    if dt:
        # warped time-step requested; use derivative
        return tw, dt * 12*(1-s)*t**2 + 12*(s-1)*t + (3-2*s)
    return tw

from functools import partial

parab = lambda x: 4*(x-0.5)**2 # curve shape
d_parab = lambda x: 8*(x-0.5) # derivative
ds = lambda x: torch.sqrt(1 + d_parab(x)**2) # differential arc length

def calculate_total_arc_length(n=1000):
    """Calculate the total arc length of the parabola y = 4(x - 0.5)**2 from x=0 to x=1"""
    x_values = torch.linspace(0, 1, n)
    arc_length_values = ds(x_values)
    total_arc_length = torch.trapz(arc_length_values, x_values)
    return total_arc_length

def fake_velocity_model(loc, t, speed=1.0):
    """For demo purposes only: Follow a parabolic path and move at unit speed
    Compute the x and y components of the velocity along the parabola y = 4(x - 0.5)^2"""
    x, y = loc[:, 0], loc[:, 1]
    slope = d_parab(x)
    direction = torch.stack([torch.ones_like(slope), slope], dim=1)
    magnitude = torch.norm(direction, dim=1, keepdim=True)
    unit_velocity = direction / magnitude
    return unit_velocity*speed

```

```

@torch.no_grad()
def integrate_motion_along_parabola(
    model, initial_points, n_steps=30, step_fn=fwd_euler_step, s=0.5,):
    """one-off integrator used only for this one visualization figure. don't use for anything else"""
    current_points = initial_points.clone()
    trajectories = [current_points.cpu().clone()]
    ts = torch.linspace(0,1.0, n_steps)
    ts = warp_time(ts, s=s) # here's the time warpage
    speed = calculate_total_arc_length() # Total travel time is 1.0 so speed "=" distance
    scaled_model = partial(model, speed=speed)
    with torch.no_grad():
        for i in range(n_steps-1):
            current_points = step_fn( scaled_model , current_points.clone(), ts[i], ts[i+1]-ts[i])
            trajectories.append(current_points.cpu().clone())
    return torch.stack(trajectories)

@torch.no_grad()
def viz_parabola_with_steps(step_fn=fwd_euler_step, n_steps=28):
    """varies warp parameter s and integrates along a parabola"""
    plt.close()
    t_curve = torch.linspace(0,1,100)

    n_t_points = n_steps # 28 if step_fn==fwd_euler_step else 6
    t_points = torch.linspace(0,1,n_t_points)

    n_s = 6 # number of different s values to show
    fig, ax = plt.subplots(1, n_s, figsize=(n_s*2.8, 3))
    plt.suptitle(f"Integration scheme = {step_fn.__name__}", fontsize=16, y=1.05)
    initial_points = torch.tensor([[0,1]]) # one point in the top left
    for i, s in enumerate(torch.linspace(.25, 1.5, n_s)): # warp time by different amounts via s parameter
        ax[i].plot(t_curve, parab(t_curve)) # solid line showing path
        traj = integrate_motion_along_parabola(fake_velocity_model, initial_points, n_steps=n_t_points,
            s=s, step_fn=step_fn).squeeze()
        err_str = f"\nerror={F.mse_loss(parab(traj[:,0]),traj[:,1]):.3g}"
        ax[i].scatter(traj[:,0], traj[:,1], label=f's = {s:.2f}{err_str}', color=(wong_cmap*2)[i])

        legend = ax[i].legend(loc='upper center', frameon=False, markerscale=0, handlelength=0, fontsize=12)
        for text in legend.get_texts():
            text.set_ha('center')

        if abs(s-1.0) < 1e-3: ax[i].set_title('Even Spacing')

    ax[0].set_title('More Points in Middle')
    ax[-1].set_title('More Points at Ends')
    plt.show()
    plt.close()

viz_parabola_with_steps()

```

A.12 RK4 step RK4 单步积分

```

def rk4_step(f, # function that takes (y,t) and returns dy/dt, i.e. velocity
            y, # current location
            t, # current t value
            dt, # requested time step size
            ):
    k1 = f(y, t)
    k2 = f(y + dt*k1/2, t + dt/2)
    k3 = f(y + dt*k2/2, t + dt/2)
    k4 = f(y + dt*k3, t + dt)
    return y + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)

viz_parabola_with_steps(step_fn=rk4_step, n_steps=6)

```

A.13 code 快速检查流的端点

```

#| code-fold: true
#| code-summary: "quick check of how our flow endpoints are looking"
pretrained_model = fm_model
pretrained_model.eval()
reflow_targets = integrate_path(pretrained_model, val_points.to(device), n_steps=8, step_fn=rk4_step,

```

```

warp_fn=warp_time).cpu()

fig, ax = plt.subplots(1,3, figsize=(10,3))
for i, [data, color, label] in enumerate(zip([val_points, reflow_targets, target_samples],
                                             [source_color, pred_color, target_color],
                                             ['Source Data', 'Learned Flow Endpoints', 'True Target Data'])):
    ax[i].scatter(data[:,0], data[:,1], color=color, label=label, alpha=0.6)
    ax[i].set_aspect('equal')
    ax[i].set_title(label)
plt.show()
plt.close()

```

A.14 snippet 目标数据的来源 (Reflow 的关键改动)

```

## target_samples = create_target_data(batch_size)          # Previous "random pairing"
target_samples = integrator(pretrained_model, source_samples) # Reflow!

```

A.15 new training loop w/ ReFlowed targets 使用 Reflow 目标的新训练循环

```

def train_reflow_model(model, pretrained_model=None,
                       n_epochs=40, lr=0.001, batch_size=2048,
                       status_every=1, viz_every=1, # in epochs
                       new_points_every=1, # in steps
                       warp_fn=warp_time,
                       step_fn=rk4_step, # rk4 so we get high-quality outputs while reflowing
                       ):
    """This is almost IDENTICAL to the previous training routine.
    The difference is the change in "target_samples" via what the RF authors call "ReFlow":
    Instead of (randomly) paring source points with points in the "true target distribution",
    we use the pretrained/teacher model to integrate the source points to their (predicted) flow endpoints
    and use THOSE as the "target" values.
    """
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()
    step, n_steps = 0, 100
    device = next(model.parameters()).device

    for epoch in range(n_epochs):
        model.train()
        pbar = tqdm(range(n_steps), leave=False)
        for _ in pbar:
            step += 1
            optimizer.zero_grad()

            if step % new_points_every == 0: # you could in theory not draw new points with each step, though we will.
                source_samples = create_source_data(batch_size).to(device)
                if pretrained_model: # HERE is the ReFlow operation...
                    target_samples = integrate_path(pretrained_model, source_samples, step_fn=rk4_step,
                                                    warp_fn=warp_time, n_steps=20)
                else:
                    target_samples = create_target_data(batch_size) # this function also supports fm models from
                    scratch

            t = torch.rand(source_samples.size(0), 1).to(device) # random times for training
            if warp_fn: t = warp_fn(t) # time warp here (different from use in integrator!) helps focus "coverage"
            i.e. sampling the space

            interpolated_samples = source_samples * (1 - t) + target_samples * t
            v = model(interpolated_samples, t)
            line_directions = target_samples - source_samples
            loss = loss_fn(v, line_directions)

            loss.backward()
            optimizer.step()

            pbar.set_description(f'Epoch [{epoch + 1}/{n_epochs}], Loss: {loss.item():.4g}')

        if (epoch + 1) % viz_every == 0:
            model.eval()
            clear_output(wait=True) # Clear previous plots
            viz(val_points, target_samples[:val_points.shape[0]], model) # don't need rk4 for reflow'd model viz b/c

```

```

paths, r, straight
    plt.show()
    plt.close() # Close the figure to free memory
    model.train()

return model

```

A.16 snippet 学生模型直接沿用教师模型的结构

```

import copy

# Note that the student/reflow model could have a simpler architecture
# than the teacher/pretrained model, but... we'll just keep 'em the same :shrug:
reflowed_model = copy.deepcopy(pretrained_model) # no need to start from scratch, use teacher's weights
reflowed_model.train() # make sure we have gradients turned on

reflowed_model = train_reflow_model(reflowed_model, pretrained_model=pretrained_model)

```

A.17 snippet 制作粒子动画对比视频

```

#| code-fold: true
#| code-summary: "Making movie for particle animation comparison"
rect_eval = reflowed_model.eval()

anim_file = "images/particles_fm_vs_rf.mp4"
create_flow_animation(val_points.clone(), models=[pretrained_model, reflowed_model],
    n_frames=50, titles=['Flow Matching', 'Reflowed Flow'], save_file=anim_file)

```

A.18 streamline animation 流线动画

```

@torch.no_grad()
def create_streamline_animation(start_dist, model, model2=None, n_frames=50, show_points=False, titles=None,
    step_fn=fwd_euler_step, # euler's ok for reflowed model bc/paths are straight
    save_file=None,
    ):
    """Create an animation showing distribution flow with streamplot background"""
    device = next(model.parameters()).device
    figsize = [5,5]
    if titles is None:
        titles = ['Flow Matching']
        if model2: titles += ['Reflowed Model']
    if model2:
        figsize[0] *= 2
    n_plots = 1 + (model2 is not None)
    fig, ax = plt.subplots(1, n_plots, figsize=figsize)
    if n_plots==1: ax = [ax]
    plt.close()

    end_dist, trajectories = integrate_path(model, start_dist.clone().to(device), n_steps=n_frames, step_fn=step_fn,
warp_fn=warp_time, save_trajectories=True)
    scatter = ax[0].scatter([], [], alpha=0.6, s=10, color=wong_pink, zorder=1)
    if model2:
        _, trajectories2 = integrate_path(model2, start_dist.clone().to(device), n_steps=n_frames, step_fn=step_fn,
warp_fn=warp_time, save_trajectories=True)
        scatter2 = ax[1].scatter([], [], alpha=0.6, s=10, color=wong_pink, zorder=1)

    max_range = max( abs(start_dist).max().item(), abs(end_dist).max().item() )

    for i in range(len(ax)):
        ax[i].set_xlim((-max_range, max_range))
        ax[i].set_ylim((-max_range, max_range))
        ax[i].set_aspect('equal')
        if titles: ax[i].set_title(titles[i])

    # Create grid for streamplot
    grid_dim = 50
    x = np.linspace(-max_range, max_range, grid_dim)
    y = np.linspace(-max_range, max_range, grid_dim)

```

```

X, Y = np.meshgrid(x, y)

# Convert grid to torch tensor for model input
grid_points = torch.tensor(np.stack([X.flatten(), Y.flatten()], axis=1), dtype=torch.float32).to(device)
color = wong_pink if show_points else (0,0,0,0)

dt = 1.0 / n_frames

def init():
    for i in range(len(ax)):
        ax[i].clear()
        ax[i].set_xlim((-max_range, max_range))
        ax[i].set_ylim((-max_range, max_range))
        scatter.set_offsets(np.c_[[], []])
    if model2:
        scatter.set_offsets(np.c_[[], []])
        return (scatter, scatter2)
    return (scatter,)

def animate(frame):
    for i in range(len(ax)):
        ax[i].clear()
        ax[i].set_xlim((-max_range, max_range))
        ax[i].set_ylim((-max_range, max_range))
        if titles: ax[i].set_title(titles[i])
        ax[i].set_xticks([])
        ax[i].set_yticks([])
        for spine in ['top', 'right', 'bottom', 'left']:
            ax[i].spines[spine].set_visible(False)

    # Update scatter plot
    current = trajectories[frame]
    scatter = ax[0].scatter(current[:, 0], current[:, 1], alpha=0.6, s=10, color=color, zorder=1)
    if model2:
        current2 = trajectories2[frame]
        scatter2 = ax[1].scatter(current2[:, 0], current2[:, 1], alpha=0.6, s=10, color=color, zorder=1)

    # Calculate vector field for current time
    t = torch.ones(grid_points.size(0), 1) * (frame * dt)
    t = warp_time(t).to(device)
    velocities = model(grid_points, t).cpu()
    U = velocities[:, 0].reshape(X.shape)
    V = velocities[:, 1].reshape(X.shape)

    x_points = np.linspace(-max_range, max_range, 15)
    y_points = np.linspace(-max_range, max_range, 15)
    X_arrows, Y_arrows = np.meshgrid(x_points, y_points)
    start_points = np.column_stack((X_arrows.ravel(), Y_arrows.ravel()))
    ax[0].streamplot(X, Y, U.numpy(), V.numpy(),
        density=5, # Controls line spacing
        color=line_color, # (0, 0, 1, 0.7),
        linewidth=0.8, maxlength=0.12,
        start_points=start_points, # This should give more arrows along paths
        arrowsize=1.2,
        arrowstyle='->')

    if model2:
        velocities2 = model2(grid_points, t).cpu()
        U2 = velocities2[:, 0].reshape(X.shape)
        V2 = velocities2[:, 1].reshape(X.shape)
        start_points2 = np.column_stack((X_arrows.ravel(), Y_arrows.ravel()))
        ax[1].streamplot(X, Y, U2.numpy(), V2.numpy(),
            density=5, # Controls line spacing
            color=line_color, # (0, 0, 1, 0.7),
            linewidth=0.8, maxlength=0.12,
            start_points=start_points2, # This should give more arrows along paths
            arrowsize=1.2,
            arrowstyle='->')

    # Update particle positions
    t = torch.ones(current.size(0), 1) * (frame * dt)
    t, dtw = warp_time(t, dt=dt)
    velocity = model(current.to(device), t.to(device)).cpu()
    current = current + velocity * dtw
    if model2:
        velocity2 = model2(current2.to(device), t.to(device)).cpu()

```

```

        current2 = current2 + velocity2 * dtw
        return (scatter, scatter2,)
    return (scatter,)

anim = animation.FuncAnimation(fig, animate, init_func=init,
                               frames=n_frames, interval=20, blit=False)

if save_file:
    anim.save(save_file, writer='ffmpeg', fps=30)
    return HTML(f"""<center><video height="350" controls loop><source src="{save_file}" type="video/mp4">
        Your browser does not support the video tag.</video></center>""")
else:
    rc('animation', html='jshtml')
    return HTML(anim.to_jshtml())

save_file = 'images/fm_vs_rf_streamvecs.mp4'
create_streamline_animation(val_points, fm_model, model2=reflowed_model, n_frames=50, save_file=save_file)#,
show_points=True)

```

Flow With What You Know — Scott H. Hawley, ICLR Blogposts 2025 | 本 PDF 由原文离线归档并中英对照重排，仅供个人学习使用。
所有文字、图片、代码版权归原作者所有。